Adjoint Matching: Fine-tuning Flow and Diffusion Generative Models with Memoryless Stochastic Optimal Control

Carles Domingo-Enrich[1], **Michal Drozdzal**[1], **Brian Karrer**[1], **Ricky T. Q. Chen**[1]

[1]FAIR, Meta

Dynamical generative models that produce samples through an iterative process, such as Flow Matching and denoising diffusion models, have seen widespread use, but there have not been many theoretically-sound methods for improving these models with reward fine-tuning. In this work, we cast reward fine-tuning as stochastic optimal control (SOC). Critically, we prove that a very specific *memoryless* noise schedule must be enforced during fine-tuning, in order to account for the dependency between the noise variable and the generated samples. We also propose a new algorithm named *Adjoint Matching* which outperforms existing SOC algorithms, by casting SOC problems as a regression problem. We find that our approach significantly improves over existing methods for reward fine-tuning, achieving better consistency, realism, and generalization to unseen human preference reward models, while retaining sample diversity.

Correspondence: Carles Domingo-Enrich at cd2754@nyu.edu

Figure 1 We introduce Adjoint Matching, a theoretically-driven yet simple algorithm for reward fine-tuning that works for a large family of dynamical generative models, including for the first time, Flow Matching models. Text prompts: "*Beautiful colorful sunset midst of building in Bangkok Thailand*", "*Beautiful grandma and granddaughter are mixing salad and smiling while cooking in kitchen*", "*The beautiful young woman in sunglasses is standing at the background of field and hill. She is smiling and looking over shoulder*", "*Chess, intellectual games, figure horse, chess board*".

1 Introduction

Flow Matching (Lipman et al., 2023; Albergo and Vanden-Eijnden, 2023; Liu et al., 2023) and denoising diffusion (Song and Ermon, 2019; Ho et al., 2020; Song et al., 2021b; Kingma et al., 2021) models are being used for many generative modeling applications, including text-to-image (Rombach et al., 2022; Esser et al., 2024), text-to-video (Singer et al., 2022), and text-to-audio (Le et al., 2024; Vyas et al., 2023). In most cases, the base generative model does not achieve the desired sample quality. To improve the generated samples, it is common to resort to techniques such as classifier-free guidance (Ho and Salimans, 2022; Zheng et al., 2023) to get better text-to-sample alignment, or to fine-tune using human preference reward models to improve sample quality and realism (Wallace et al., 2023a; Clark et al., 2024).

In the adjacent field of large language models, the behavior of the model is aligned to human preferences through fine-tuning with reinforcement learning from human feedback (RLHF). Either explicitly or implicitly, RLHF methods (Ziegler et al., 2020; Stiennon et al., 2020; Ouyang et al., 2022; Bai et al., 2022) assume a reward model $r(x)$ that captures human preferences, with the goal of modifying the base generative model such that it generates the following *tilted distribution*:

$$p^*(x) \propto p^{\text{base}}(x) \exp(r(x)), \tag{1}$$

where p_{base} is the base generative model's sample distribution.

Inspired by this, fine-tuning methods have been developed to improve denoising diffusion models based on human preference data; either using a reward-based approach (Fan and Lee, 2023; Black et al., 2024; Fan et al., 2023; Xu et al., 2023; Clark et al., 2024; Uehara et al., 2024a,b), or direct preference optimization (Wallace et al., 2023a). However, unlike the fine-tuning methods designed for large language models, most of the existing methods to a large degree ignore p^{base} and focus solely on the reward model. Reward models can range from standard evaluation metrics such as ClipScore (Hessel et al., 2021; Kirstain et al., 2023) to specialized models that have been trained on human preferences (Schuhmann and Beaumont, 2022; Xu et al., 2023; Wu et al., 2023c). As these are parameterized by neural networks, they fall pray to adversarial examples which lead to the generation of undesirable artifacts (Goodfellow et al., 2014; Mordvintsev et al., 2015). This has led some works to consider adding regularization during fine-tuning (Fan et al., 2024; Uehara et al., 2024b) to incentivize staying close to the base model distribution; however, there does not yet exist a *simple* approach which actually provably generates from the tilted distribution (1).

The main contributions of our paper are as follows:

(i) We present a stochastic optimal control (SOC) formulation for reward fine-tuning of dynamical generative models. Importantly, we prove that the naïve approach considered by prior works lead to a *value function bias* problem that biases the fine-tuned model away from the tilted distribution (1). This problem has also been observed by Uehara et al. (2024b) but they propose a more complicated solution which involves training a separate generative model for the optimal noise distribution.

(ii) Instead, we propose a very simple solution: the *memoryless noise schedule*. This is a unique noise schedule that completely removes the dependency between noise variables and the generated samples, resulting in provable convergence to the tilted distribution. This allows us to fine-tune dynamical generative models in full generality, including being the first to fine-tune noiseless Flow Matching models.

(iii) We also propose a new method for solving SOC problems, called *Adjoint Matching*, which combines the scalability of gradient-based methods and the simplicity of a least-squares regression objective. This is orthogonal to the reward fine-tuning application and can be applied to general SOC problems.

(iv) We perform extensive comparisons to baseline approaches, and analyze them from multiple perspectives such as realism, consistency, and diversity. We find that our proposed method provides generalization to unseen human preference reward models, better text-to-sample consistency, and retains good diversity.

In the following, sections are broken down as follows: Section 2 summarizes the algorithms used for sampling from pre-trained Flow Matching and diffusion models, while Section 3 provides a common notation that we will use throughout. Sections 4 and 5 form the core of our contributions. Section 4 details the value function bias problem and our proposed solution via the memoryless noise schedule. Section 5 details the new Adjoint Matching algorithm for solving SOC problems.

2 Preliminaries on dynamical generative models

We are interested in fine-tuning base generative models $p^{\text{base}}(X_1)$ where samples are generated through the simulation of a stochastic process. That is, these models transform noise variables into a sample through an iterative process. In particular, we discuss the specific constructions and sampling processes of Flow Matching (Lipman et al., 2023; Liu et al., 2023; Liu, 2022; Albergo and Vanden-Eijnden, 2023) and Denoising Diffusion Models (Ho et al., 2020; Song et al., 2021b,a). The goal of this section is to provide background information on these methods, which we will later unify into a single consistent notation in Section 3.

Given random variables from an initial distribution $\bar{X}_0 \sim p_0 = \mathcal{N}(0, I)$, and $\bar{X}_1$ which are distributed according to some data distribution, we define the reference flow $\bar{\boldsymbol{X}} = (\bar{X}_t)_{t\in[0,1]}$ where

$$\bar{X}_t = \beta_t \bar{X}_0 + \alpha_t \bar{X}_1, \tag{2}$$

where $(\alpha_t)_{t\in[0,1]}, (\beta_t)_{t\in[0,1]}$ are functions such that $\alpha_0 = \beta_1 = 0$ and $\alpha_1 = \beta_0 = 1$. Diffusion models and Flow Matching construct generative Markov processes X_t with initial distribution $X_0 \sim \mathcal{N}(0, I)$ that result in flows $\boldsymbol{X} = (X_t)_{t\in[0,1]}$ with the same time marginals as the reference flow $\bar{\boldsymbol{X}}$, *i.e.*, the random variables X_t and $\bar{X}_t$ have identical distribution for all times $t \in [0, 1]$. This implies X_1 has the same distribution as the data distribution, so simulating the Markov process from random noise X_0 is a way to generate artificial samples[1].

2.1 Flow Matching

In its simplest form, the generative Markov process of a Flow Matching model is an ordinary differential equation (ODE) of the form:

$$\mathrm{d}X_t = v(X_t, t)\,\mathrm{d}t, \qquad X_0 \sim \mathcal{N}(0, I). \tag{3}$$

where $v(X_t, t)$ is a parametric velocity that is optimized to match the derivative of the reference flow, *i.e.*, $v(X_t, t) = \operatorname{argmin}_{\hat{v}} \mathbb{E}\left\|\hat{v}(\bar{X}_t, t) - \frac{\mathrm{d}}{\mathrm{d}t}\bar{X}_t\right\|^2$ (see *e.g.* Lipman et al. (2023) for details on pre-training Flow Matching models). It can then be proven that the solution of the generative process (3) has the same time marginals as the reference flow (Lipman et al., 2023; Liu, 2022; Albergo and Vanden-Eijnden, 2023), and a commonly used choice is $\alpha_t = t$ and $\beta_t = 1 - t$. One can also consider a family of stochastic differential equations (SDEs) with an arbitrary state-independent diffusion coefficient[2]:

$$\mathrm{d}X_t = \left(v(X_t, t) + \frac{\sigma(t)^2}{2\beta_t(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t - \dot{\beta}_t)}\left(v(X_t, t) - \frac{\dot{\alpha}_t}{\alpha_t}X_t\right)\right)\mathrm{d}t + \sigma(t)\,\mathrm{d}B_t, \qquad X_0 \sim \mathcal{N}(0, I), \tag{4}$$

where $(B_t)_{t\geq 0}$ is a Brownian motion. The generative processes in (3) and (4) have the same time marginals. This can be seen by writing down the Fokker-Planck equations for (3) and (4), and observing that they are the same up to a cancellation of terms (Maoutsa et al., 2020). The diffusion coefficient $\sigma(t)$ in (4) is compensated by the second term in the drift which scales proportionally as $\sigma(t)^2$.

2.2 Denoising Diffusion Models

We next discuss diffusion models, in particular the sampling scheme proposed by Denoising Diffusion Implicit Model (DDIM; Song et al. (2021a)) which we will later relate to Denoising Diffusion Probabilistic Models (DDPM; Ho et al. (2020)) as a particular case of the former. For sampling from a diffusion model, the DDIM update rule[3] (Song et al. (2021a), Eq. 12), typically stated in discrete time with $k \in \{0, \dots, K\}$, is:

$$X_{k+1} = \sqrt{\bar{\alpha}_{k+1}}\left(\frac{X_k - \sqrt{1-\bar{\alpha}_k}\epsilon(X_k, k)}{\sqrt{\bar{\alpha}_k}}\right) + \sqrt{1 - \bar{\alpha}_{k+1} - \sigma_k^2}\epsilon(X_k, k) + \sigma_k \varepsilon_k, \qquad \varepsilon_k \sim \mathcal{N}(0, I),\ X_0 \sim \mathcal{N}(0, I), \tag{5}$$

where $\bar{\alpha}_k$ is an increasing sequence such that $\bar{\alpha}_0 = 0$, $\bar{\alpha}_K = 1$, and the sequence σ_k is arbitrary. That is, one samples an initial Gaussian random variable x_0, and applies the stochastic update (5) iteratively K times in order to obtain an artificial sample X_K. Updates can be interpreted as progressively denoising the iterate: x_0 is completely noisy and x_K is fully denoised. The noise predictor model $\epsilon(x_k, k)$ is trained to predict the noise of x_k (see *e.g.* Ho et al. (2020) for details on pre-training denoising diffusion models).

[1] In our derivations, we will simply assume the base model has been trained perfectly during the pre-training phase.

[2] We use the common short-hand "over-dot" notation to denote the time derivative, *i.e.*, $\dot{x}_t = \frac{\mathrm{d}}{\mathrm{d}t}x_t$.

[3] We slightly depart from the notation in Song et al. (2021a) by flipping the direction of time and using $\bar{\alpha}_k$ which corresponds to the α_k in Song et al. (2021a) while it corresponds to the $\bar{\alpha}_k$ in Ho et al. (2020).

3 Flow Matching and diffusion models from a common perspective

We formulate Flow Matching and diffusion models in a unified framework, which we will later use throughout the paper. Firstly, to simplify notation, we will be using continuous-time formulations. This will also directly enable fine-tuning methods inspired by the continuous-time paradigm, which we find tends to perform better than discrete-time counterparts in our empirical validations. Secondly, by consolidating notation, we will be able to discuss fine-tuning of dynamical generative models that follow the same time marginals as the reference flow (2), pre-trained with either the Denoising Diffusion or Flow Matching framework, in full generality.

To convert DDIM to a continuous-time stochastic process, we can show that the DDIM update rule (5), up to a first-order approximation, is equivalent to the Euler-Maruyama discretization of the following SDE:

$$\mathrm{d}X_t = \big(\tfrac{\dot{\bar{\alpha}}_t}{2\bar{\alpha}_t} X_t - \big(\tfrac{\dot{\bar{\alpha}}_t}{2\bar{\alpha}_t} + \tfrac{\sigma(t)^2}{2}\big)\tfrac{\epsilon^{\text{base}}(X_t,t)}{\sqrt{1-\bar{\alpha}_t}}\big)\mathrm{d}t + \sigma(t)\mathrm{d}B_t, \qquad X_0 \sim \mathcal{N}(0, I). \tag{6}$$

See Appendix B.1 for the full derivation. To go from (5) to (6), we assumed a uniform discretization of time, *i.e.* $t = \frac{k}{K}$. This results in identifying the discrete-time process $(X_k)_{k\in\{0,\dots,K\}}$ with a continuous-time process $(X_t)_{t\in[0,1]}$, where $\bar{\alpha}_k := \bar{\alpha}_t$, $\sigma_k := \frac{1}{\sqrt{K}}\sigma(t)$, and $\epsilon(X_k, k)$ with $\epsilon^{\text{base}}(X_k, t)$. In relation to the reference flow (2), the generative process in (6) has the same time marginals when $\alpha_t = \sqrt{\bar{\alpha}_t}$ and $\beta_t = \sqrt{1-\bar{\alpha}_t}$ (Ho et al., 2020).

Furthermore, when viewed up to first order approximations, the DDPM sampling scheme (Ho et al. (2020); Algorithm 2) can be seen as special instance of the DDIM sampling scheme when $\sigma(t) = \sqrt{\dot{\bar{\alpha}}_t/\bar{\alpha}_t}$. This results in the following generative process:

$$\mathrm{d}X_t = \big(\tfrac{\dot{\bar{\alpha}}_t}{2\bar{\alpha}_t} X_t - \tfrac{\dot{\bar{\alpha}}_t}{\bar{\alpha}_t}\tfrac{\epsilon^{\text{base}}(X_t,t)}{\sqrt{1-\bar{\alpha}_t}}\big)\mathrm{d}t + \sqrt{\tfrac{\dot{\bar{\alpha}}_t}{\bar{\alpha}_t}}\mathrm{d}B_t, \qquad X_0 \sim \mathcal{N}(0, I), \tag{7}$$

We can further consolidate notation by converting all quantities to the score function $\mathfrak{s}(x,t)$—defined as the gradient of the log density of the random variable X_t—which is possible when X_0 is Normal-distributed and under the affine reference flow (2). In particular, the velocity v^{base} from Flow Matching can be expressed in terms of the score function (see Appendix B.4):

$$v^{\text{base}}(x,t) = \tfrac{\dot{\alpha}_t}{\alpha_t} x + \beta_t(\tfrac{\dot{\alpha}_t}{\alpha_t}\beta_t - \dot{\beta}_t)\mathfrak{s}(x,t). \tag{8}$$

And the noise predictor ϵ^{base} also admits an expression in terms of the score function (see Appendix B.3):

$$\mathfrak{s}(x,t) = -\tfrac{\epsilon^{\text{base}}(x,t)}{\sqrt{1-\bar{\alpha}_t}}. \tag{9}$$

Plugging these two equations into (4) and (6), respectively, and rewriting them in terms of only the α_t and β_t in (2), we can unify both the Flow Matching and continuous-time DDIM generative processes as:

$$\mathrm{d}X_t = b(X_t, t)\,\mathrm{d}t + \sigma(t)\,\mathrm{d}B_t, \qquad X_0 \sim \mathcal{N}(0, I), \tag{10}$$

$$\text{where } b(x,t) = \kappa_t x + \big(\tfrac{\sigma(t)^2}{2} + \eta_t\big)\mathfrak{s}(x,t), \quad \kappa_t = \tfrac{\dot{\alpha}_t}{\alpha_t}, \quad \eta_t = \beta_t(\tfrac{\dot{\alpha}_t}{\alpha_t}\beta_t - \dot{\beta}_t) \tag{11}$$

where (α_t, β_t) are coefficients of the reference flow (2). We have hence expressed the generative process of a base model, whether it is a Flow Matching or a diffusion model, as an SDE of the form (10)-(11), unified by the choice of reference flow. This expression has been written before for DDIM, e.g. Bartosh et al. (2024a,b).

4 Fine-tuning as "memoryless" stochastic optimal control

We now discuss the crux of the problem: how to produce a fine-tuned generative model that produces samples X_1 which follow the tilted distribution involving a reward model (1). An obvious direction is to construct a *fine-tuning objective* involving both the base generative model and the reward model, where the optimal solution results in a fine-tuned generative model for the tilted distribution. However, as we will explain, this turns out to be non-trivial, because a naïve formulation will introduce bias into the solution.

In Section 4.1, we discuss the problem formulation of stochastic optimal control, a general framework for optimizing SDEs, and its relation to the maximum entropy reinforcement learning framework commonly used

for RLHF fine-tuning. Next, in Section 4.2, we discuss the *initial value function bias* problem which plagues existing approaches and so far has seen no simple solution. Finally, in Section 4.3, we propose a novel simple solution that circumvents the bias problem, by enforcing a particular diffusion coefficient, the *memoryless noise schedule*, to be used during fine-tuning. This results in an extremely simple fine-tuning objective that provably converges to a model which generates the tilted distribution (1) without any statistical bias.

4.1 Preliminaries on the stochastic optimal control problem formulation

Stochastic optimal control (SOC; Bellman (1957); Fleming and Rishel (2012); Sethi (2018)) considers general optimization problems over stochastic differential equations, but we only need to consider a common instantiation, the quadratic cost control-affine problem formulation:

$$\min_{u\in\mathcal{U}} \mathbb{E}\Big[\int_0^1 \left(\tfrac{1}{2}\|u(X_t^u,t)\|^2 + f(X_t^u,t)\right)\mathrm{d}t + g(X_1^u)\Big], \tag{12}$$

$$\text{s.t. } \mathrm{d}X_t^u = \left(b(X_t^u,t) + \sigma(t)u(X_t^u,t)\right)\,\mathrm{d}t + \sigma(t)\mathrm{d}B_t, \qquad X_0^u \sim p_0 \tag{13}$$

where in (13), $X_t^u \in \mathbb{R}^d$ is the state of the stochastic process, $u : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ is commonly referred to as the control vector field, $b : \mathbb{R}^d \times [0,1] \to \mathbb{R}^d$ is a base drift, and $\sigma : [0,1] \to \mathbb{R}^{d\times d}$ is the diffusion coefficient. These jointly define the *controlled process* $\boldsymbol{X}^u \sim p^u$ that we are interested in optimizing; often both b and σ are fixed and we only optimize over the control u.

As part of the objective functional (12), we have an affine control cost $\frac{1}{2}\|u(X_t^u,t)\|^2$, a running state cost $f : \mathbb{R}^d \times [0,1] \to \mathbb{R}$ and a terminal state cost $g : \mathbb{R}^d \to \mathbb{R}$.

The stochastic optimal control (SOC) objective (12) can be decomposed recursively from the final time value. It is common to define the *cost functional* which is the expected future cost starting from state x at time t:

$$J(u;x,t) := \mathbb{E}_{\boldsymbol{X}\sim p^u}\left[\int_t^1 \left(\tfrac{1}{2}\|u(X_s,s)\|^2 + f(X_s,s)\right)\,\mathrm{d}s + g(X_1) \;\middle|\; X_t = x\right]. \tag{14}$$

From here, the *value function* is the optimal value of the cost functional[4] :

$$V(x,t) := \min_{u\in\mathcal{U}} J(u;x,t) = J(u^*;x,t), \tag{15}$$

where u^* is the *optimal control*, *i.e.*, minimizer of (12). Furthermore, a classical result is that the value function can be expressed in terms of the *uncontrolled* base process p^{base} (Kappen (2005), see Domingo-Enrich et al. 2023, Eq. 8, App. B for a self-contained proof):

$$V(x,t) = -\log \mathbb{E}_{\boldsymbol{X}\sim p^{\text{base}}}\left[\exp(-\int_t^1 f(X_s,s)\mathrm{d}s - g(X_1)) \;\middle|\; X_t = x\right]. \tag{16}$$

A useful expression for the optimal control (which we will make use of in deriving the Adjoint Matching objective in Section 5) is that it is related to the gradient of the value function:

$$u^*(x,t) = -\sigma(t)^\top \nabla_x V(x,t) = -\sigma(t)^\top \nabla_x J(u^*,x,t). \tag{17}$$

Relation to MaxEnt RL. Stochastic optimal control with the control-affine formulation (12) is the continuous-time equivalence of maximum entropy reinforcement learning (MaxEnt RL; Todorov (2006); Ziebart et al. (2008)) with a KL regularization instead of only an entropy regularization. In particular, by the Girsanov theorem (Theorem 2), the affine control cost is equivalent to a Kullback–Leibler (KL) divergence between the base process p^{base}, when $u = 0$, and the controlled process p^u, when conditioned on the same initial state X_0 (see Appendix C.4):

$$D_{\mathrm{KL}}\big(p^u(\boldsymbol{X}|X_0) \;\big\|\; p^{base}(\boldsymbol{X}|X_0)\big) = \mathbb{E}_{\boldsymbol{X}^u\sim p^u}\left[\int_0^1 \tfrac{1}{2}\|u(X_t^u,t)\|^2\mathrm{d}t\right], \tag{18}$$

resulting in the KL-regularized RL interpretation of (12):

$$\max_{u\in\mathcal{U}} \mathbb{E}_{X_0\sim p_0}\left[\mathbb{E}_{\boldsymbol{X}\sim p^u(\cdot|X_0)}\left[\int_0^1 -f(X_t^u,t)\mathrm{d}t - g(X_1^u)\right] - D_{\mathrm{KL}}(p^u(\boldsymbol{X}|X_0) \;\|\; p^{base}(\boldsymbol{X}|X_0))\right], \tag{19}$$

where the negative state costs correspond to intermediate and terminal rewards in the RL interpretation. The KL divergence incentivizes the optimal solution to stay close to the distribution of the base process.

[4]Note that there is a slight difference in terminology between SOC and reinforcement learning, where our cost functional is referred to as the state value function and our value function is the optimal state value function in RL.

4.2 The initial value function bias problem

We next discuss why naïvely adding a KL regularization does not lead to the tilted distribution (1). From (19), we can also show that the optimal distribution conditioned on X_0 is[5]

$$p^*(\boldsymbol{X}|X_0) \propto p^{\text{base}}(\boldsymbol{X}|X_0) \exp\big(-\int_0^1 f(X_t,t)\,\mathrm{d}t - g(X_1)\big). \tag{20}$$

This is analogous to the exponentiated reward distribution in MaxEnt RL (Rawlik et al., 2013), but since we generalize the entropy regularization to a KL regularization, p^{base} acts as a prior distribution.

In order to relate this to the tilted distribution (1) that we want to achieve for fine-tuning, first notice that the normalization constant of the right-hand side (RHS) of (20) is exactly the value function at $t = 0$:

$$\mathbb{E}_{\boldsymbol{X}\sim p^{\text{base}}(\boldsymbol{X}|X_0)}\left[\exp\big(-\int_0^1 f(X_t,t)\,\mathrm{d}t - g(X_1)\big)\right] = \exp\big(-V(X_0,0)\big)\,, \tag{21}$$

where the equality is due to (16). Dividing the RHS of (20) by (21) and multiplying by $p_0(X_0)$, we obtain the normalized distribution over the full path $\boldsymbol{X}$,

$$p^*(\boldsymbol{X}) = p^{\text{base}}(\boldsymbol{X}) \exp\big(-\int_0^1 f(X_t,t)\,\mathrm{d}t - g(X_1) + V(X_0,0)\big). \tag{22}$$

Setting $f = 0$ and $g = -r$, we arrive at an expression for the optimal distribution

$$p^*(X_0,X_1) = p^{\text{base}}(X_0,X_1) \exp\big(r(X_1) + V(X_0,0)\big). \tag{23}$$

This unfortunately does not lead to the tilted distribution (1) because we have a bias in the optimal distribution that is due to the value function of the initial distribution $V(X_0, 0)$. That is to say, naïvely adding a KL regularization (18) to the fine-tuning objective in the sense of (19) leads to a biased distribution (22) after fine-tuning and is *not* equivalent to the tilted distribution (1). For instance, when the sampling procedure is noiseless, *i.e.*, $\sigma(t) = 0$, fine-tuning naïvely will not have any effect because X_0 completely determines X_1.

This is unlike the situation for large language models (Ouyang et al., 2022; Rafailov et al., 2023), where there is no dynamical process that samples X_1 iteratively and hence no dependence on the initial noise variable X_0. Although this KL regularization is a common objective for RLHF of large language models, it has seen seldom use in fine-tuning diffusion models, likely due to this issue of the initial value function bias.

In the context of diffusion models, KL regularization (19) has been explored in prior works (Fan et al., 2024), but its behavior was not well-understood and they did not relate the fine-tuned model to the tilted distribution (1). Another direction that has been proposed is to learn the initial distribution p_0 to cancel out the bias (Uehara et al., 2024b; Tang, 2024) but this simply shifts the work into tilting the initial distribution and requires an auxiliary model for parameterizing the optimal initial distribution. In contrast, we show in the next section that it is possible to remove the value function bias by simply choosing a very particular noise schedule during the fine-tuning procedure.

4.3 The memoryless noise schedule for fine-tuning dynamical generative models

In this section, we propose a very simple method of turning (23) into the tilted distribution (1) through the use of a particular *memoryless* noise schedule. Throughout, we provide an intuitive explanation of why this noise schedule is sufficient for fine-tuning while discussing the full theoretical result where we show that the memoryless noise schedule is actually not only sufficient but also necessary.

Intuitively, the main reason we cannot arrive at the tilted distribution from (23) is due to the $p^{\text{base}}(X_0, X_1)$ distribution not factoring into X_0 and X_1. Hence, we define a memoryless generative process as follows:

Definition 1 (Memoryless generative process)**.** *A generative process of the form* (10)-(11) *is memoryless if* X_0 *and* X_1 *are independent, i.e.,* $p^{base}(X_0, X_1) = p^{base}(X_0)p^{base}(X_1)$.

[5]Note (20) is informal because densities over continuous-time processes are ill-defined; the formal statement is $\frac{\mathrm{d}\mathbb{P}^*}{\mathrm{d}\mathbb{P}^{\text{base}}}(\boldsymbol{X}|X_0) = \exp(-\int_0^1 f(X_t,t)\,\mathrm{d}t - g(X_1))$, where $\frac{\mathrm{d}\mathbb{P}^*}{\mathrm{d}\mathbb{P}^{\text{base}}}$ denotes the Radon-Nikodym derivative. We treat this formally in the proofs.

	κ_t	η_t	Diffusion coefficient $\sigma(t)$	Memoryless X_t
Flow Matching (3)	$\frac{\dot\alpha_t}{\alpha_t}$	$\beta_t\big(\frac{\dot\alpha_t}{\alpha_t}\beta_t - \dot\beta_t\big)$	General (commonly 0)	No
Memoryless Flow Matching (4)	$\frac{\dot\alpha_t}{\alpha_t}$	$\beta_t\big(\frac{\dot\alpha_t}{\alpha_t}\beta_t - \dot\beta_t\big)$	$\sqrt{2\eta_t}$	Yes
DDIM (6)	$\frac{\dot{\bar\alpha}_t}{2\bar\alpha_t}$	$\frac{\dot{\bar\alpha}_t}{2\bar\alpha_t}$	General (commonly 0)	No
DDPM (7)	$\frac{\dot{\bar\alpha}_t}{2\bar\alpha_t}$	$\frac{\dot{\bar\alpha}_t}{2\bar\alpha_t}$	$\sqrt{2\eta_t}$	Yes

Table 1 Diffusion coefficient $\sigma(t)$ and the factors κ_t, η_t for the Flow Matching, Memoryless Flow Matching, DDIM, and DDPM generative processes. When the diffusion coefficient is $\sigma(t) = \sqrt{2\eta_t}$, the generative process is memoryless, *i.e.*, samples X_1 will be independent of the initial noise X_0.

When the base generative process is memoryless, this implies:

$$p^*(X_1) = \int p^{\text{base}}(X_0)p^{\text{base}}(X_1)\exp(r(X_1) + V(X_0, 0))\mathrm{d}X_0 \propto p^{\text{base}}(X_1)\exp(r(X_1)). \tag{24}$$

That is, solving the SOC problem (12)-(13) with a memoryless base model will result in a fine-tuned model that generates samples $p^*(X_1)$ according to the tilted distribution (1). This memoryless property is not satisfied generally by the family of generative processes captured by (12)-(13). For instance, the Flow Matching and DDIM generative processes with zero diffusion coefficient (*i.e.*, $\sigma(t) = 0$) are definitely not memoryless due to X_0 and X_1 being theoretically invertible. Below, we provide the sufficient and neccessary condition for the noise schedule in order to have a memoryless generative process.

Proposition 1 (Memoryless noise schedules)**.** *Within the family of generative processes* (10)-(11)*, a generative process is memoryless if and only if the noise schedule is chosen as:*

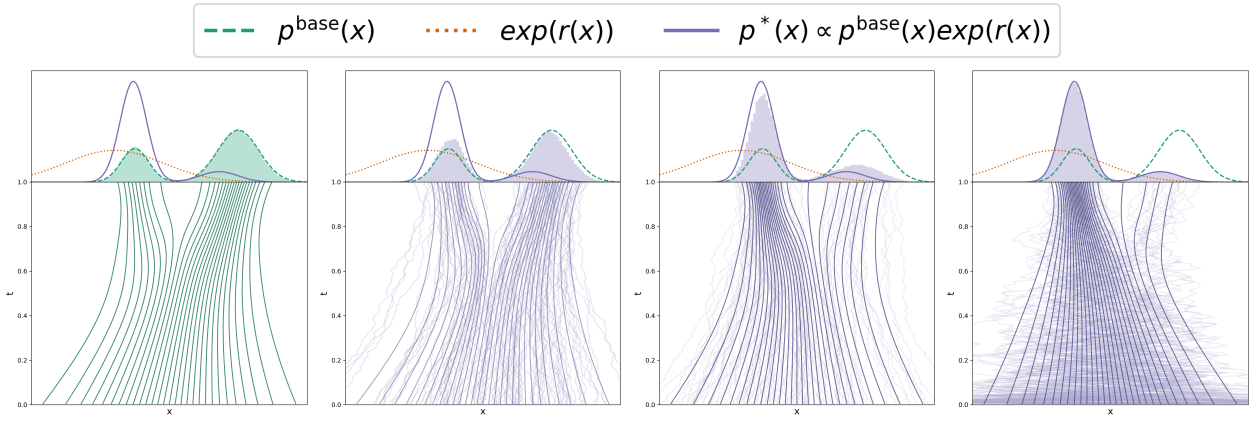
$$\sigma(t)^2 = 2\eta_t + \chi(t), \;\; \textit{where } \chi : [0,1] \to \mathbb{R} \textit{ is s.t. } \forall t \in (0,1], \quad \lim_{t'\to 0^+} \alpha_{t'} \exp\big(-\textstyle\int_{t'}^{t} \frac{\chi(s)}{2\beta_s^2}\,\mathrm{d}s\big) = 0. \tag{25}$$

where η_t is the coefficient defined in (11) *(see also Table 1). In particular, we refer to $\sigma(t) = \sqrt{2\eta_t}$ as the memoryless noise schedule.*

Due to the endpoint constraints of (α_t, β_t) for the reference flow (2), the memoryless noise schedule $\sigma(t)$ is infinite at $t = 0$ and approaches zero at $t = 1$. This provides a way for the generative process to mix when close to noise X_0 while stay steadying when close to the sample X_1. Hence, the sample will have no information about X_0 due to the enormous amount of mixing with a large diffusion coefficient. Furthermore, while we have intuitively justified the memoryless noise schedule through its independence property, our theoretical result is actually even stronger: all generative models of the form (10)-(11) *must* be fine-tuned using the memoryless noise schedule. We formalize this in the following theorem, which we prove in Appendix D.2:

Theorem 1 (Fine-tuning recipe for general noise schedule sampling)**.** *Within the family of generative processes* (10)-(11)*, in order to allow the use of arbitrary noise schedules and still generate samples according to the tilted distribution* (1)*, the fine-tuning problem* (12)-(13) *with $f = 0$ and $g = -r$ must be done with the memoryless noise schedule $\sigma(t) = \sqrt{2\eta_t}$.*

Theorem 1 states that we *need* to use the memoryless noise schedule for fine-tuning with the SOC objective—or equivalently, the KL regularized reward objective (19). This is the only noise schedule that retains the relationship between the velocity and score function, allowing the conversion to arbitrary noise schedules (*e.g.*, $\sigma(t) = 0$) after fine-tuning. It is worth noting that when using the memoryless noise schedule for DDIM, this recovers what we derived as the continuous-time limit of the DDPM generative process (7). However, the DDPM sampler (Ho et al., 2020) is not commonly used while the DDIM sampler (Song et al., 2021a) and Flow Matching models typically generate samples using $\sigma(t) = 0$, so an explicit conversion to the memoryless noise schedule is necessary for fine-tuning. To the best of our knowledge, we are not aware of any existing works that have proposed a time-varying diffusion coefficient with theoretical guarantees. Table 1 summarizes the memoryless schedule for diffusion and Flow Matching models, which we refer to as Memoryless Flow Matching. In Figure 2, we visualize fine-tuning a 1D model, where we see that constant $\sigma(t)$ leads to biased distributions whereas the memoryless noise schedule perfectly converges to the tilted distribution (1).



(a) Pre-trained FM v^{base} **(b)** Fine-tuned FM v^{finetune} with $\sigma(t) = 0.2$ **(c)** Fine-tuned FM v^{finetune} with $\sigma(t) = 1.0$ **(d)** Fine-tuned FM v^{finetune} with memoryless $\sigma(t) = \sqrt{2\eta_t}$

Figure 2 Visualization of Theorem 1 showing that fine-tuning must be done with the memoryless noise schedule to ensure convergence to the tilted distribution (1). (a) Shows the base Flow Matching model. (b, c) Fine-tuning using a constant $\sigma(t)$ leads to biased distributions. (d) Fine-tuning using the memoryless noise schedule leads to the correct tilted distribution. Note that sample generation can use any noise schedule after fine-tuning, including $\sigma(t) = 0$.

For convenience, we plug the memoryless noise schedule into the controlled process for fine-tuning (13), and express them in terms of each respective framework. Let ϵ^{base}, v^{base} denote the pre-trained vector fields and $\epsilon^{\text{finetune}}$, v^{finetune} the fine-tuned vector fields. Then we have the following expressions for the full drift $b(x,t) + \sigma(t)u(x,t)$ and control $u(x,t)$ when $\sigma(t) = \sqrt{2\eta_t}$:

DDIM / DDPM:

$$b(x,t) + \sigma(t)u(x,t) = \frac{\dot{\bar{\alpha}}_t}{2\bar{\alpha}_t}x - \frac{\dot{\bar{\alpha}}_t}{\bar{\alpha}_t}\frac{\epsilon^{\text{finetune}}(x,t)}{\sqrt{1-\bar{\alpha}_t}}, \qquad u(x,t) = -\sqrt{\frac{\dot{\bar{\alpha}}_t}{\bar{\alpha}_t(1-\bar{\alpha}_t)}}(\epsilon^{\text{finetune}}(x,t) - \epsilon^{\text{base}}(x,t)). \tag{26}$$

Memoryless Flow Matching:

$$b(x,t) + \sigma(t)u(x,t) = 2v^{\text{finetune}}(x,t) - \frac{\dot{\alpha}_t}{\alpha_t}x, \qquad u(x,t) = \sqrt{\frac{2}{\beta_t(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t - \dot{\beta}_t)}}(v^{\text{finetune}}(x,t) - v^{\text{base}}(x,t)). \tag{27}$$

Thus, to solve the SOC problem (12)-(13) in practice, we parameterize the control u in terms of $\epsilon^{\text{finetune}}$ or v^{finetune} and optimize these vector fields instead. After plugging in (26)-(27), the SOC problem (12)-(13) can then be solved using any SOC algorithm in order to perform fine-tuning, and we proposed an especially effective algorithm next in Section 5. After fine-tuning, $\epsilon^{\text{finetune}}$ and v^{finetune} can simply be plugged back into their respective generative processes (3)-(7) to sample from the tilted distribution (1) using any choice of diffusion coefficient.

5 Adjoint Matching for control-affine stochastic optimal control

We discuss existing methods and also propose a new method for optimizing control-affine SOC problems. The new Adjoint Matching method is a combination of the time-tested continuous adjoint method (Pontryagin, 1962) with recent developments on constructing least-squares objectives for solving SOC problems (Domingo-Enrich et al., 2023). In this section, we briefly discuss preliminaries on existing methods, their pros and cons, then detail the Adjoint Matching algorithm and its surprising connections to the prior methods. For numerical optimization, we now assume that the control u is a parametric model with parameters θ.

5.1 Existing methods for stochastic optimal control

5.1.1 The adjoint method

The most basic method of optimizing the simulation of an SDE is to directly differentiate through the simulation using gradients from the SOC objective function (Han and E, 2016). The adjoint method simply uses the objective:

$$\mathcal{L}(u; \boldsymbol{X}) := \int_0^1 \left(\tfrac{1}{2}\|u(X_t, t)\|^2 + f(X_t, t)\right) \mathrm{d}t + g(X_1), \qquad \boldsymbol{X} \sim p^u. \tag{28}$$

This is a stochastic estimate of the control objective in (12), and the goal is to take compute the gradient of $\mathcal{L}(u; \boldsymbol{X})$ with respect to the parameters θ of the control u. Due to the continuous-time nature of SDEs, there are two main approaches to implementing this numerically. Firstly, the *Discrete Adjoint* method uses a "discretize-then-differentiate" approach, where the numerical solver for simulating the SDE is simply stored in memory then differentiated through, and it has been studied extensively (*e.g.*, Bierkens and Kappen (2014); Gómez et al. (2014); Hartmann and Schütte (2012); Kappen et al. (2012); Rawlik et al. (2013); Haber and Ruthotto (2017)). This approach, however, uses an extremely large amount of memory as the full computational graph of the numerical solver must be stored in memory and implementations often must rely on gradient checkpointing (Chen et al., 2016) to reduce memory usage.

Secondly, the *Continuous Adjoint* method exploits the continuous-time nature of SDEs and uses an analytical expression for the gradient of the control objective with respect to the intermediate states X_t, expressed as an adjoint ODE, and then applies a numerical method to simulate this gradient itself, hence it is referred to as a "differentiate-then-discretize" approach (Pontryagin, 1962; Chen et al., 2018; Li et al., 2020). We first define the *adjoint state* as:

$$\begin{aligned} &a(t; \boldsymbol{X}, u) := \nabla_{X_t}\Big(\int_t^1 \left(\tfrac{1}{2}\|u(X_{t'}, t')\|^2 + f(X_{t'}, t')\right) \mathrm{d}t' + g(X_1)\Big), \\ &\text{where } \boldsymbol{X} \text{ solves } \mathrm{d}X_t = \big(b(X_t, t) + \sigma(t)u(X_t, t)\big)\ \mathrm{d}t + \sigma(t)\mathrm{d}B_t. \end{aligned} \tag{29}$$

This implies that $\mathbb{E}_{\boldsymbol{X}\sim p^u}\left[a(t; \boldsymbol{X}, u) \mid X_t = x\right] = \nabla_x J(u; x, t)$, where J denotes the cost functional defined in (14). It can then be shown that this adjoint state satisfies [6]:

$$\frac{\mathrm{d}}{\mathrm{d}t}a(t; \boldsymbol{X}, u) = -\left[a(t; \boldsymbol{X}, u)^\mathsf{T}\left(\nabla_{X_t}(b(X_t, t) + \sigma(t)u(X_t, t))\right) + \nabla_{X_t}\left(f(X_t, t) + \tfrac{1}{2}\|u(X_t, t)\|^2\right)\right], \tag{30}$$

$$a(1; \boldsymbol{X}, u) = \nabla g(X_1). \tag{31}$$

The adjoint state is solved backwards in time, starting from the terminal condition (31). Computation of (30) can be efficiently done as a vector-Jacobian product on automatic differentiation software (Paszke et al., 2019). Once the adjoint state has been solved for $t \in [0, 1]$, then the gradient of $\mathcal{L}(u; \boldsymbol{X})$ with respect to the parameters θ can be obtained by integrating over the entire time interval:

$$\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\theta} = \tfrac{1}{2}\int_0^1 \frac{\partial}{\partial\theta}\|u(X_t, t)\|^2 \mathrm{d}t + \int_0^1 \frac{\partial u(X_t, t)}{\partial\theta}^\mathsf{T} \sigma(t)^\mathsf{T} a(t; \boldsymbol{X}, u)\mathrm{d}t, \tag{32}$$

where the first term is the partial derivative of $\mathcal{L}$ w.r.t. θ and the second term is the partial derivative through the sample trajectory $\boldsymbol{X}$. See Proposition 6 in Appendix E.1 for a statement and proof of this result. The discrete and continuous adjoint methods converge to the same gradient as the step size of the numerical solvers go to zero. Both are scalable to high dimensions and have seen their fair share of usage in optimizing neural ODE/SDEs (Chen et al., 2018, 2021; Li et al., 2020). As the adjoint methods are essentially gradient-based optimization algorithms applied on a highly non-convex problem, many have also reported they can be unstable empirically (Mohamed et al., 2020; Suh et al., 2022; Domingo-Enrich et al., 2023).

5.1.2 Importance-weighted matching objectives for regressing onto the optimal control

An alternative is to consider regressing onto the optimal control u^*, which is the approach of the cross-entropy method (Rubinstein and Kroese, 2013; Zhang et al., 2014) and stochastic optimal control matching (SOCM; Domingo-Enrich et al. (2023)). These methods make use of path integral theory (Kappen, 2005) to express

[6] Note we use the convention that a Jacobian matrix $J = \nabla_x v(x)$ is defined as $J_{ij} = \frac{\partial v_i(x)}{\partial x_j}$.

the optimal control through importance sampling, resulting in an *importance-weighted* least-squares objective function

$$\mathcal{L}_{\text{SOCM}}(u; \boldsymbol{X}) := \int_0^1 \|u(X_t, t) - \hat{u}^*(X_t, t)\|^2 \mathrm{d}t \times \omega(u, \boldsymbol{X}), \qquad \boldsymbol{X} \sim p^u, \tag{33}$$

where ω is an importance weighting that approximates sampling from the optimal distribution p^*, and $\hat{u}^*$ is a stochastic estimator of the optimal control relying on having sampled from the optimal process. We defer to Domingo-Enrich et al. (2023) for the exact details. The functional landscape of this objective is convex, which is argued to help yield stable training. However, the need for importance sampling renders this impractical for high dimensional applications: the variance of the importance weighting ω grows exponentially with dimension of the stochastic process, leading to catastrophic failure. This unfortunately means that such importance-weighted matching objectives are impractical for fine-tuning dynamical generative models; however, a least-squares objective is greatly coveted as it can lead to stable training and simple interpretations.

5.2 Adjoint Matching

We make two important observations which lead to our proposed method: (*i*) it is possible to construct a matching objective without any importance weighting, and (*ii*) there are unnecessary terms in the adjoint differential equation (30) that can lead to higher variance at convergence.

Firstly, we notice that we can simply match the gradient of the cost functional under the *current* control. That is, while SOCM carefully constructs an importance-weighted estimator of the *optimal* control $u^* = -\sigma(t)^\mathsf{T} \nabla J(u^*; x, t)$ (17), we claim that we can actually just regress onto the target vector field $-\sigma(t)^\mathsf{T} \nabla J(u; x, t)$ where u is the current control, and furthermore, this results in a gradient equal in expectation to the continuous adjoint method. We formalize this in the following proposition, proven in Appendix E.2:

Proposition 2. *Let us define, for now, the basic Adjoint Matching objective as:*

$$\mathcal{L}_{\text{Basic}-\text{Adj}-\text{Match}}(u; \boldsymbol{X}) := \tfrac{1}{2} \int_0^1 \left\| u(X_t, t) + \sigma(t)^\mathsf{T} a(t; \boldsymbol{X}, \bar{u}) \right\|^2 \mathrm{d}t, \qquad \boldsymbol{X} \sim p^{\bar{u}}, \quad \bar{u} = \texttt{stopgrad}(u), \tag{34}$$

where $\bar{u} = \texttt{stopgrad}(u)$ means that the gradients of $\bar{u}$ with respect to the parameters θ of the control u are artificially set to zero. The gradient of $\mathcal{L}_{\text{Basic}-\text{Adj}-\text{Match}}(u; \boldsymbol{X})$ with respect to θ is equal to the gradient $\frac{\mathrm{d}\mathcal{L}}{\mathrm{d}\theta}$ in equation (32). Importantly, the only critical point of $\mathbb{E}\left[\mathcal{L}_{\text{Basic}-\text{Adj}-\text{Match}}\right]$ is the optimal control u^*.

Critical points of $\mathcal{L}$ are controls u such that $\frac{\delta}{\delta u}\mathcal{L}(u) = 0$, where $\frac{\delta}{\delta u}\mathcal{L}$ denotes the first variation of the functional $\mathcal{L}$. In other words, Proposition 2 states that the only control that satisfies the first-order optimality condition for the basic Adjoint Matching objective is the optimal control, which provides theoretical grounding for gradient-based optimization algorithms.

An intuitive way to understand the basic Adjoint Matching objective is that it is a *consistency loss*. The Adjoint Matching objective is based off of the observation that the optimal control $u^*(x, t)$ is the unique fixed-point of the relation $u(x, t) = -\sigma(t)^\mathsf{T} \nabla_x J(u; x, t)$ (see Lemma 6 in Appendix E.2) and so we are directly optimizing for a control that fits this relation, while using the adjoint state as a stochastic estimator of $\nabla_x J(u; x, t)$ (29).

The basic Adjoint Matching objective in Proposition 2 does not yet yield a novel algorithm for stochastic optimal control, because it produces the same gradient as the continuous adjoint method. This can be seen by taking the gradient w.r.t. θ after expanding the square in (34) and removing terms that do not depend on θ to arrive exactly at the continuous adjoint method (32). However, it provides the means of deriving a simpler *leaner* objective function.

The "Lean" Adjoint. The minimizer of a least-squares objective is the conditional expectation of the regression target, so for the Adjoint Matching objective, at the optimum we have that

$$u^*(x, t) = \mathbb{E}_{\boldsymbol{X} \sim p^*} \left[-\sigma(t)^\mathsf{T} a(t; \boldsymbol{X}, u^*) | X_t = x \right]. \tag{35}$$

Multiplying both sides by the Jacobian $\nabla_x u^*(x, t)$ and re-arranging, we get the relation

$$\mathbb{E}_{\boldsymbol{X} \sim p^*} \left[u^*(x, t)^\mathsf{T} \nabla_x u^*(x, t) + a(t; \boldsymbol{X}, u^*)^\mathsf{T} \sigma(t) \nabla_x u^*(x, t) \mid X_t = x \right] = 0. \tag{36}$$

Algorithm 1 Adjoint Matching for fine-tuning Flow Matching models

Input: Pre-trained FM velocity field v^{base}, step size h, number of fine-tuning iterations N.
Initialize fine-tuned vector fields: $v^{\text{finetune}} = v^{\text{base}}$ with parameters θ.
for $n \in \{0, \dots, N-1\}$ **do**

Sample m trajectories $\boldsymbol{X} = (X_t)_{t\in\{0,\dots,1\}}$ with memoryless noise schedule $\sigma(t) = \sqrt{2\beta_t(\frac{\dot\alpha_t}{\alpha_t}\beta_t - \dot\beta_t)}$, *e.g.*:

$$X_{t+h} = X_t + h\left(2v_\theta^{\text{finetune}}(X_t,t) - \tfrac{\dot\alpha_t}{\alpha_t}X_t\right) + \sqrt{h}\sigma(t)\varepsilon_t, \qquad \varepsilon_t \sim \mathcal{N}(0,I), \qquad X_0 \sim \mathcal{N}(0,I). \tag{40}$$

For each trajectory, solve the *lean adjoint ODE* (38)-(39) backwards in time from $t = 1$ to 0, *e.g.*:

$$\tilde a_{t-h} = \tilde a_t + h\tilde a_t^\intercal \nabla_{X_t}\left(2v^{\text{base}}(X_t,t) - \tfrac{\dot\alpha_t}{\alpha_t}X_t\right), \qquad \tilde a_1 = -\nabla_{X_1} r(X_1). \tag{41}$$

Note that X_t and $\tilde a_t$ should be computed without gradients, *i.e.*, $X_t = \texttt{stopgrad}(X_t)$, $\tilde a_t = \texttt{stopgrad}(\tilde a_t)$.

For each trajectory, compute the Adjoint Matching objective (37):

$$\mathcal{L}_{\text{Adj}-\text{Match}}(\theta) = \textstyle\sum_{t\in\{0,\dots,1-h\}} \left\| \frac{2}{\sigma(t)}\left(v_\theta^{\text{finetune}}(X_t,t) - v^{\text{base}}(X_t,t)\right) + \sigma(t)\tilde a_t\right\|^2. \tag{42}$$

Compute the gradient $\nabla_\theta \mathcal{L}(\theta)$ and update θ using favorite gradient descent algorithm.
end
Output: Fine-tuned vector field v^{finetune}

Notice that the terms inside the expectation in (36) show up as part of the adjoint differential equation (30), which we have now shown to have expectation zero at the optimal solution. Therefore, we motivate the definition of a *lean adjoint state* $\tilde a$ with the terms in (36) removed. Plugging this lean adjoint back into the least-squares objective, we obtain our final proposed Adjoint Matching objective:

$$\mathcal{L}_{\text{Adj}-\text{Match}}(u;\boldsymbol{X}) := \tfrac{1}{2}\textstyle\int_0^1 \left\|u(X_t,t) + \sigma(t)^\intercal \tilde a(t;\boldsymbol{X})\right\|^2 \mathrm{d}t, \qquad \boldsymbol{X}\sim p^{\bar u}, \quad \bar u = \texttt{stopgrad}(u), \tag{37}$$

$$\text{where} \quad \tfrac{\mathrm{d}}{\mathrm{d}t}\tilde a(t;\boldsymbol{X}) = -(\tilde a(t;\boldsymbol{X})^\top \nabla_x b(X_t,t) + \nabla_x f(X_t,t)), \tag{38}$$

$$\tilde a(1;\boldsymbol{X}) = \nabla_x g(X_1). \tag{39}$$

Equations (38)-(39) define the *lean adjoint state*, and (37) is the complete Adjoint Matching objective. *The unique critical point of* $\mathbb{E}[\mathcal{L}_{\text{Adj}-\text{Match}}]$ *is the optimal control*, which we prove relying on Proposition 2 and equation (36) (see Proposition 7 in Appendix E.3).

Compared to the importance sampling methods (Section 5.1.2), Adjoint Matching is a simple least-squares regression objective and has no importance weighting. This allows it to avoid the pitfalls of high variance importance weights and makes it as scalable as the adjoint methods while retaining the interpretation of matching a target vector field.

Compared to the adjoint method (Section 5.1.1), Adjoint Matching produces a *different gradient in expectation than the continuous adjoint*. This is because the lean adjoint state is not related to the gradient of the cost functional anymore, *i.e.*, (29) is not true, except at the optimum when $u = u^*$. Even at the optimal solution, since Adjoint Matching removes terms that have expectation zero, it can potentially exhibit better convergence and lower variance than the continuous adjoint method. Additionally, computation of the lean adjoint state (38) also exhibits a smaller computational cost due to the removal of the extra terms (no longer need the Jacobian of the control $\nabla_x u$). We provide a rigorous derivation of Adjoint Matching and the above claims in Appendix E.3.

Adjoint Matching can be applied to reward fine-tuning of dynamical generative models through the memoryless SOC formulation discussed in Section 4. We provide pseudo-code for this in Algorithm 1 for Flow Matching models and in Algorithm 2 in Appendix E.4 for denoising diffusion models.

6 Related work

Fine-tuning from human feedback. There are two main overarching approaches to RLHF: the *reward-based* approach (Ziegler et al., 2020; Stiennon et al., 2020; Ouyang et al., 2022; Bai et al., 2022) and *direct preference optimization* (DPO; Rafailov et al. (2023)). The reward-based approach (Ziegler et al., 2020; Stiennon et al., 2020; Ouyang et al., 2022; Bai et al., 2022) consists in learning the reward model $r(x)$ from human preference data, and then solving a maximum entropy RL problem with rewards produced by $r(x)$. DPO merges the two previous steps into one: there is no need to learn $r(x)$ as human preference data is directly used to fine-tune the model. However, DPO is typically only applied with a filtered dataset, and does not work explicitly with a reward model. Furthermore, for flow and diffusion models specifically, it is possible to differentiate the reward function, so there is a larger emphasis on reward-based approaches.

Fine-tuning for diffusion models. Among existing reward-based diffusion fine-tuning methods, Fan and Lee (2023) interpret the denoising process as a multi-step decision-making task and use policy gradient algorithms to fine-tune diffusion samplers. Black et al. (2024) makes use of proximal policy gradients for fine-tuning but this does not make use of the differentiability of the reward model. Fan et al. (2023) also consider KL-regularized rewards (19) but do not make the critical connection to the tilted distribution (1) that we flesh out in Section 4.2. The fine-tuning algorithms of Xu et al. (2023); Clark et al. (2024) directly take gradients of the reward model and use heuristics to try to stay close to the original base generative model, but their behavior is not well understood and unrelated to the tilted distribution: Xu et al. (2023) takes gradients of the reward applied on the denoised sample at different points in time, and Clark et al. (2024) backpropagates the reward function through all or part of the diffusion trajectory. Finally, Uehara et al. (2024b) also fine-tune diffusion models with the goal of sampling from the tilted distribution (1), but their approach is much more involved than ours as it requires learning a value function, and solving two stochastic optimal control problems. Additional reward fine-tuning works include Bruna and Han (2024), that provide theoretical guarantees to sample from the tilted distribution when the reward is a quadratic function, and Zhang et al. (2024), that propose a reward fine-tuning algorithm for the GFlowNet architecture.

Inference-time optimization methods. Some have proposed methods that do not update the base model but instead modify the generation process directly. One approach is to add a guidance term to the velocity (Chung et al., 2022; Song et al., 2023; Pokle et al., 2023); however, this is a heuristic and it is not well-understood what particular distribution is being generated. Another approach is to directly optimize the initial noise distribution (Li, 2021; Wallace et al., 2023b; Ben-Hamu et al., 2024); this is taking an opposite approach to the inital value bias problem than us by moving all of the work into optimizing the initial distribution. A more computationally intensive approach is to perform online estimation of the optimal control, for the purpose of heuristically solving an optimal control problem within the sampling process (Huang et al., 2024; Rout et al., 2024); these approaches aim to solve a separate control problem for each generated sample, instead of performing amortization (Amos et al., 2023) to learn a fine-tuned generative model.

Optimal control in generative modeling. Methods from optimal control have been used to train dynamical generative models parameterized by ODEs (Chen et al., 2018), SDEs (Li et al., 2020), and jump processes (Chen et al., 2021), enabled through the adjoint method. They can be used to train arbitrary generative processes, but for simplified constructions these have fallen in favor of simulation-free matching objectives such as denoising score matching (Vincent, 2011) and Flow Matching (Lipman et al., 2023). The optimal control formalism also has significance in sampling from un-normalized distributions (Zhang and Chen, 2022; Berner et al., 2023; Vargas et al., 2023, 2022; Richter and Berner, 2024; Tzen and Raginsky, 2019). The inclusion of a state cost has been used to solve transport problems where intermediate path distributions are of importance (Liu et al., 2024; Pooladian et al., 2024). These collective advances naturally lead to the consideration of the optimal control formalism for reward fine-tuning.

Conditional sampling in inverse problems. Denker et al. (2024) and Wu et al. (2023a) independently consider a pre-trained diffusion model $p(x)$, and an observation y on the generated sample x, as well as the analytic likelihood $p(y|x)$. Their aim is to sample from the posterior $p(x)p(y|x)$, and their applications include inpainting, class-conditional generation, super-resolution, phase retrieval, non-linear deblurring, computed

	Fine-tuning Method	Fine-tuning $\sigma(t)$	Sampling $\sigma(t)$	ClipScore↑	PickScore↑	HPS v2↑	DreamSim Diversity↑
	None (Base model)	N/A	$\sqrt{2\eta_t}$	$24.15_{\pm 0.26}$	$17.25_{\pm 0.06}$	$16.19_{\pm 0.17}$	$53.60_{\pm 1.37}$
			0	$28.32_{\pm 0.22}$	$18.15_{\pm 0.07}$	$17.89_{\pm 0.16}$	**56.53**$_{\pm 1.52}$
Baselines	DRaFT-1	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$30.18_{\pm 0.24}$	$19.38_{\pm 0.08}$	$24.61_{\pm 0.17}$	$25.54_{\pm 0.99}$
		0	0	$30.95_{\pm 0.28}$	$19.37_{\pm 0.06}$	$24.37_{\pm 0.17}$	$27.39_{\pm 1.14}$
	DRaFT-40	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$26.94_{\pm 0.28}$	$18.34_{\pm 0.19}$	$19.98_{\pm 1.02}$	$41.98_{\pm 2.14}$
		0	0	$30.07_{\pm 0.39}$	$19.45_{\pm 0.08}$	$24.06_{\pm 0.24}$	$36.53_{\pm 1.69}$
	DPO	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$24.11_{\pm 0.22}$	$17.24_{\pm 0.06}$	$16.15_{\pm 0.14}$	$53.27_{\pm 1.36}$
		0	0	$27.77_{\pm 0.18}$	$17.92_{\pm 0.07}$	$17.30_{\pm 0.20}$	$54.11_{\pm 1.50}$
	ReFL	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$28.59_{\pm 0.31}$	$18.68_{\pm 0.10}$	$22.24_{\pm 0.46}$	$32.71_{\pm 2.76}$
		0	0	$30.06_{\pm 0.63}$	$19.07_{\pm 0.21}$	$23.06_{\pm 0.41}$	$32.69_{\pm 1.28}$
Memoryless SOC	Cont. Adjoint $\lambda = 12500$	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$26.99_{\pm 0.43}$	$18.33_{\pm 0.16}$	$20.83_{\pm 0.63}$	$46.59_{\pm 1.40}$
			0	$29.49_{\pm 0.32}$	$18.98_{\pm 0.16}$	$21.34_{\pm 0.53}$	$48.41_{\pm 1.44}$
	Disc. Adjoint $\lambda = 12500$	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$28.04_{\pm 0.57}$	$18.44_{\pm 0.21}$	$20.04_{\pm 0.39}$	$54.90_{\pm 2.03}$
			0	$29.28_{\pm 0.17}$	$18.82_{\pm 0.14}$	$19.73_{\pm 0.17}$	$53.36_{\pm 2.48}$
	Adj.-Matching $\lambda = 1000$	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$30.36_{\pm 0.22}$	$19.29_{\pm 0.08}$	$24.12_{\pm 0.17}$	$40.89_{\pm 1.50}$
			0	$31.41_{\pm 0.22}$	$19.57_{\pm 0.09}$	$23.29_{\pm 0.18}$	$43.10_{\pm 1.76}$
	Adj.-Matching $\lambda = 2500$	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$30.59_{\pm 0.40}$	$19.49_{\pm 0.10}$	$24.85_{\pm 0.23}$	$37.07_{\pm 1.47}$
			0	$31.64_{\pm 0.21}$	$19.71_{\pm 0.09}$	$24.12_{\pm 0.27}$	$39.88_{\pm 1.59}$
	Adj.-Matching $\lambda = 12500$	$\sqrt{2\eta_t}$	$\sqrt{2\eta_t}$	$30.62_{\pm 0.30}$	$19.50_{\pm 0.09}$	**24.95**$_{\pm 0.28}$	$34.50_{\pm 1.33}$
			0	**31.65**$_{\pm 0.19}$	**19.76**$_{\pm 0.08}$	$24.49_{\pm 0.27}$	$37.24_{\pm 1.57}$

Table 2 Evaluation metrics of different fine-tuning methods for text-to-image generation. The second and third columns show the noise schedules $\sigma(t)$ used for fine-tuning and for sampling: $\sigma(t) = \sqrt{2\eta_t}$ corresponds to Memoryless Flow Matching, and $\sigma(t) = 0$ to the Flow Matching ODE (3). We report standard errors estimated over 3 runs of the fine-tuning algorithm on random sets of 40000 training prompts, each evaluated over a random set of 1000 test prompts.

tomography, and protein design. Their setting reduces to a particular case of our reward fine-tuning framework by setting $r(x) = \log p(y|x)$. Denker et al. (2024) formulate an SOC problem, and they solve it via the log-variance loss (Richter et al. (2020); Nüsken and Richter (2021)), and the moment loss (Nüsken and Richter, 2021)[7], which they refer to as the trajectory balance loss (Malkin et al., 2023). Wu et al. (2023a) propose Twisted Diffusion Sampler, an algorithm based on Sequential Monte Carlo that uses increased inference-time compute to reduce bias. A third work that also tackles the conditional sampling problem is Du et al. (2024), which use a Lagrangian formulation that they solve approximately using Gaussian paths.

7 Experiments

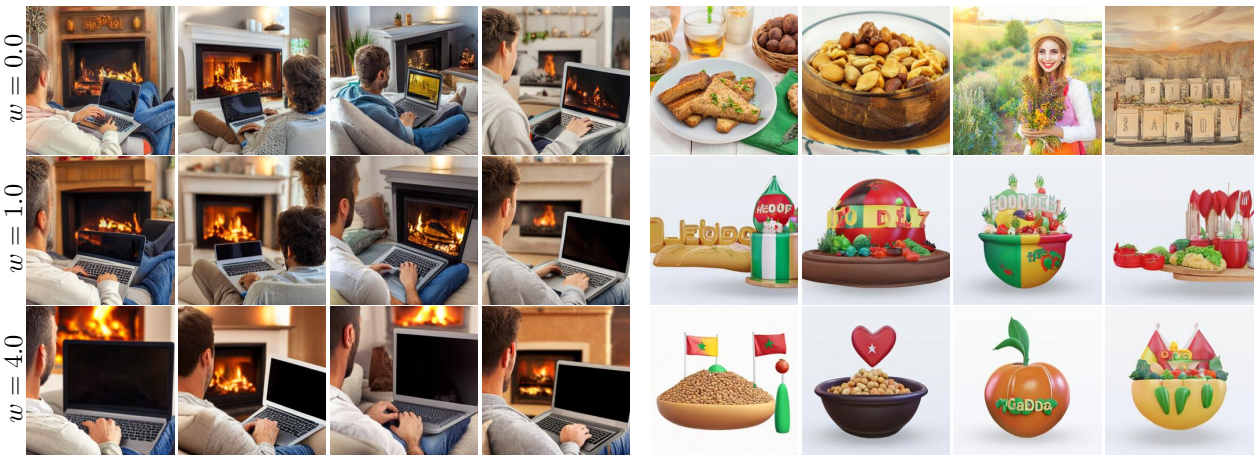
We experimentally validate our proposed method on reward fine-tuning a Flow Matching base model (Lipman et al., 2023). In particular, we use the usual setup of pre-training an autoencoder for 512×512 resolution images, then training a text-conditional Flow Matching model on the latent variables with a U-net architecture (Long et al., 2015), similar to the setup in Rombach et al. (2022). We pre-trained our base model using a dataset of licensed text and image pairs. Then for fine-tuning, we consider the reward function:

$$r(x) := \lambda \times \texttt{RewardModel}(x) \tag{43}$$

corresponding to a scaled version of the reward model, which we take to be ImageReward (Xu et al., 2023). Different values of λ provide different tradeoffs between the KL regularization and the reward model (19).

[7] See also Domingo-Enrich (2024) for a comparison among SOC losses.

Figure 3 Our proposed Adjoint Matching using the memoryless SOC formulation introduces a much more principled way of trading off how close to stay to the base model while optimizing the reward model. In contrast, baseline methods such as DRaFT-1 only optimize the reward model and must rely on early stopping to perform this trade off, resulting in a much more sensitive hyperparameter. Samples are produced using $\sigma(t) = 0$ with the same noise sample. Text prompts: "*Handsome Smiling man in blue jacket portrait*" and "*Quinoa and Feta Stuffed Baby Bell Peppers*".



Text prompt: "*Man sitting on sofa at home in front of fireplace and using laptop computer, rear view*"

Text prompt: "*3D World Food Day Morocco*"

Figure 4 Generated samples from varying classifier-free guidance weight w, from an Adjoint Matching fine-tuned model. Higher guidance increases text-to-image consistency but loses diversity and has use cases for generating highly structured images such as 3D renderings. Corresponding samples from the base model can be found in Figure 7.

For evaluation and benchmarking purposes, we report metrics that separately quantify text-to-image consistency, human preference, and sample diversity, capturing the tradeoff between each aspect of generative models (Astolfi et al., 2024). For consistency, we make use of the standard ClipScore (Hessel et al., 2021) and PickScore (Kirstain et al., 2023); for generalization to unseen human preferences, we use the HPSv2 model (Wu et al., 2023b); and for diversity, we compute averages of pairwise distances of the DreamSim features (Fu et al., 2023). More details are provided in Appendix G.4.

As our baselines, we consider the DPO (Wallace et al., 2023a), ReFL (Xu et al., 2023), and DRaFT-K algorithms (Clark et al., 2024). DPO does not use gradients from the reward function, while ReFL and DRaFT make use of heuristic gradient stopping approaches to stay close to the base generative model. Out of these baseline methods, we find that DRaFT-1 performs the best, so we perform additional ablation experiments comparing to this method. Within the same SOC formulation as our method, we also consider the

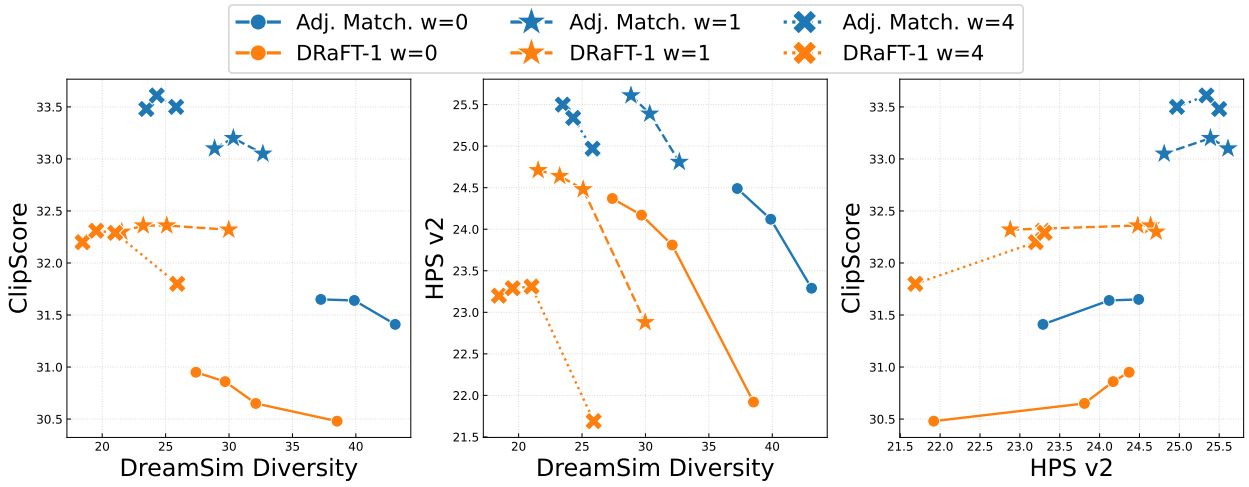


Figure 5 Tradeoffs between different aspects of generative models: text-to-image consistency (ClipScore), sample diversity for each prompt (DreamSim Diversity), and generalization to unseen human preferences (HPS v2). Different points are obtained from varying values of λ for Adjoint Matching and varying number of fine-tuning iterations for the DRaFT-1 baseline. Overall, we find our proposed method Adjoint Matching has the best Pareto fronts.

discrete and continuous adjoint methods. We provide full experimental details in Appendix G; an important implementation detail is that we slightly offset $\sigma(t)$ in order to avoid division by zero.

Evaluation results. In Table 2 we report the evaluation metrics for the baselines as well as our proposed Adjoint Matching approach. We compare each method at roughly the same wall clock time (see the times and number of iterations in Table 4, and comments in Appendix G.5). We find that across all metrics, our proposed memoryless SOC formulation outperforms existing baseline methods. The choice of SOC algorithms also obviously favors Adjoint Matching over continuous and discrete adjoint methods, which result in poorer consistency and human preference metrics.

Ablation: base model vs. reward tradeoff. We note that the scaling in front of the reward model λ determines how strongly the we should prefer the reward model over the base model. As such, we see a natural tradeoff curve: higher λ results in better consistency and human preference, but lower diversity in the generated samples. Overall, we find that Adjoint Matching performs stably across all values of λ. Our method of regularizing the fine-tuning procedure through memoryless SOC works much better than baseline methods which often must employ early stopping. We show the qualitative effect of varying λ in Figure 3, while for the DRaFT-1 baseline we show the effect of varying the number of fine-tuning iterations.

Ablation: classifier-free guidance. We note that it is possible to apply classifier-free guidance (CFG; Ho and Salimans (2022); Zheng et al. (2023)) after fine-tuning. We use the formula $(1+w)v(x,t|y) - wv(x,t)$, where w is the guidance weight, $v(x,t|y)$ is a fine-tuned text-to-image model while $v(x,t)$ is an unconditional image model. This is not principled as only the conditional model is fine-tuned, but generally it is unclear what distribution guided models sample from anyhow. In Figure 5 we show the evaluation metrics with classifier-free guidance applied. Comparing three different guidance weight values, we see a higher weight does improve text-to-image consistency, and to some extent, human preference, but this comes at the cost of being worse in terms of diversity. We show qualitative differences in Figure 4.

8 Conclusion

We investigate the problem of fine-tuning dynamical generative models such as Flow Matching and propose the use of a stochastic optimal control (SOC) formulation with a memoryless noise schedule. This ensures we converge to the same tilted distribution that the large language modeling literature uses for learning

from human feedback. In particular, the memoryless noise schedule corresponds to DDPM sampling for diffusion models and a new Memoryless Flow Matching generative process for flow models. In conjunction, we propose a novel training algorithm for solving stochastic optimal control problems, by casting SOC as a regression problem, which we call the Adjoint Matching objective. Empirically, we find that our memoryless SOC formulation works better than multiple existing works on fine-tuning diffusion models, and our Adjoint Matching algorithm outperforms related gradient-based methods. In summary, we are the first to provide a theoretically-driven algorithm for fine-tuning Flow Matching models, and we find that our approach significantly outperforms baseline methods across multiple axes of evaluation—text-to-image consistency, generalization to unseen human preference, and sample diversity—on large-scale text-to-image generation.

References

Michael S Albergo, Nicholas M Boffi, and Eric Vanden-Eijnden. Stochastic interpolants: A unifying framework for flows and diffusions. *arXiv preprint arXiv:2303.08797*, 2023. Cited on page 36.

Michael Samuel Albergo and Eric Vanden-Eijnden. Building normalizing flows with stochastic interpolants. In *The Eleventh International Conference on Learning Representations*, 2023. Cited on pages 2, 3, and 36.

Brandon Amos et al. Tutorial on amortized optimization. *Foundations and Trends® in Machine Learning*, 16(5): 592–732, 2023. Cited on page 12.

Pietro Astolfi, Marlene Careil, Melissa Hall, Oscar Mañas, Matthew Muckley, Jakob Verbeek, Adriana Romero Soriano, and Michal Drozdzal. Consistency-diversity-realism pareto fronts of conditional image generative models. *arXiv preprint arXiv:2406.10429*, 2024. Cited on page 14.

Yuntao Bai, Andy Jones, Kamal Ndousse, Amanda Askell, Anna Chen, Nova DasSarma, Dawn Drain, Stanislav Fort, Deep Ganguli, Tom Henighan, Nicholas Joseph, Saurav Kadavath, Jackson Kernion, Tom Conerly, Sheer El-Showk, Nelson Elhage, Zac Hatfield-Dodds, Danny Hernandez, Tristan Hume, Scott Johnston, Shauna Kravec, Liane Lovitt, Neel Nanda, Catherine Olsson, Dario Amodei, Tom Brown, Jack Clark, Sam McCandlish, Chris Olah, Ben Mann, and Jared Kaplan. Training a helpful and harmless assistant with reinforcement learning from human feedback. *arXiv preprint arXiv:2204.05862*, 2022. Cited on pages 2 and 12.

Grigory Bartosh, Dmitry Vetrov, and Christian A. Naesseth. Neural diffusion models. *arXiv preprint arXiv:2310.08337*, 2024a. Cited on page 4.

Grigory Bartosh, Dmitry Vetrov, and Christian A. Naesseth. Neural flow diffusion models: Learnable forward process for improved diffusion modelling. *arXiv preprint arXiv:2404.12940*, 2024b. Cited on page 4.

Richard Bellman. *Dynamic programming*. Princeton Landmarks in Mathematics. Princeton University Press, Princeton, NJ, 2010., 1957. Cited on page 5.

Heli Ben-Hamu, Omri Puny, Itai Gat, Brian Karrer, Uriel Singer, and Yaron Lipman. D-flow: Differentiating through flows for controlled generation. *arXiv preprint arXiv:2402.14017*, 2024. Cited on page 12.

Julius Berner, Lorenz Richter, and Karen Ullrich. An optimal control perspective on diffusion-based generative modeling. *arXiv preprint arXiv:2211.01364*, 2023. Cited on page 12.

Joris Bierkens and Hilbert J Kappen. Explicit solution of relative entropy weighted control. *Systems & Control Letters*, 72:36–43, 2014. Cited on page 9.

Kevin Black, Michael Janner, Yilun Du, Ilya Kostrikov, and Sergey Levine. Training diffusion models with reinforcement learning. In *The Twelfth International Conference on Learning Representations*, 2024. Cited on pages 2, 12, and 37.

Joan Bruna and Jiequn Han. Posterior sampling with denoising oracles via tilted transport. *arXiv preprint arXiv:2407.00745*, 2024. Cited on page 12.

Ricky T. Q. Chen, Yulia Rubanova, Jesse Bettencourt, and David K Duvenaud. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems*, volume 31. Curran Associates, Inc., 2018. Cited on pages 9 and 12.

Ricky T. Q. Chen, Brandon Amos, and Maximilian Nickel. Learning neural event functions for ordinary differential equations. In *International Conference on Learning Representations*, 2021. Cited on pages 9 and 12.

Tianqi Chen, Bing Xu, Chiyuan Zhang, and Carlos Guestrin. Training deep nets with sublinear memory cost. *arXiv preprint arXiv:1604.06174*, 2016. Cited on page 9.

Hyungjin Chung, Jeongsol Kim, Michael T Mccann, Marc L Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. *arXiv preprint arXiv:2209.14687*, 2022. Cited on page 12.

Kevin Clark, Paul Vicol, Kevin Swersky, and David J. Fleet. Directly fine-tuning diffusion models on differentiable rewards. In *The Twelfth International Conference on Learning Representations*, 2024. Cited on pages 2, 12, and 14.

Valentin De Bortoli, James Thornton, Jeremy Heng, and Arnaud Doucet. Diffusion schrödinger bridge with applications to score-based generative modeling. In *Advances in Neural Information Processing Systems*, volume 34, pages 17695–17709. Curran Associates, Inc., 2021. Cited on page 34.

Alexander Denker, Francisco Vargas, Shreyas Padhy, Kieran Didi, Simon Mathis, Vincent Dutordoir, Riccardo Barbano, Emile Mathieu, Urszula Julia Komorowska, and Pietro Lio. Deft: Efficient finetuning of conditional diffusion models by learning the generalised h-transform. *arXiv preprint arXiv:2406.01781*, 2024. Cited on pages 12 and 13.

Carles Domingo-Enrich. A taxonomy of loss functions for stochastic optimal control. *arXiv preprint arXiv:2410.00345*, 2024. Cited on page 13.

Carles Domingo-Enrich, Jiequn Han, Brandon Amos, Joan Bruna, and Ricky T. Q. Chen. Stochastic optimal control matching. *arXiv preprint arXiv:2312.02027*, 2023. Cited on pages 5, 8, 9, 10, 46, and 47.

Yuanqi Du, Michael Plainer, Rob Brekelmans, Chenru Duan, Frank Noé, Carla P. Gomes, Alan Apsuru-Guzik, and Kirill Neklyudov. Doob's lagrangian: A sample-efficient variational approach to transition path sampling. *arXiv preprint arXiv:2410.07974*, 2024. Cited on page 13.

Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024. Cited on page 2.

Ying Fan and Kangwook Lee. Optimizing ddpm sampling with shortcut fine-tuning. In *International Conference on Machine Learning*, 2023. Cited on pages 2 and 12.

Ying Fan, Olivia Watkins, Yuqing Du, Hao Liu, Moonkyung Ryu, Craig Boutilier, Pieter Abbeel, Mohammad Ghavamzadeh, Kangwook Lee, and Kimin Lee. Dpok: Reinforcement learning for fine-tuning text-to-image diffusion models. *arXiv preprint arXiv:2305.16381*, 2023. Cited on pages 2 and 12.

Ying Fan, Olivia Watkins, Yuqing Du, Hao Liu, Moonkyung Ryu, Craig Boutilier, Pieter Abbeel, Mohammad Ghavamzadeh, Kangwook Lee, and Kimin Lee. Reinforcement learning for fine-tuning text-to-image diffusion models. *Advances in Neural Information Processing Systems*, 36, 2024. Cited on pages 2 and 6.

W.H. Fleming and R.W. Rishel. *Deterministic and Stochastic Optimal Control*. Stochastic Modelling and Applied Probability. Springer New York, 2012. Cited on page 5.

Stephanie Fu, Netanel Tamir, Shobhita Sundaram, Lucy Chai, Richard Zhang, Tali Dekel, and Phillip Isola. Dreamsim: Learning new dimensions of human visual similarity using synthetic data. *arXiv preprint arXiv:2306.09344*, 2023. Cited on pages 14 and 54.

Vicenç Gómez, Hilbert J Kappen, Jan Peters, and Gerhard Neumann. Policy search for path integral control. In *Joint European Conference on Machine Learning and Knowledge Discovery in Databases*, pages 482–497. Springer, 2014. Cited on page 9.

Ian J Goodfellow, Jonathon Shlens, and Christian Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint arXiv:1412.6572*, 2014. Cited on page 2.

Eldad Haber and Lars Ruthotto. Stable architectures for deep neural networks. *Inverse problems*, 34(1):014004, 2017. Cited on page 9.

Jiequn Han and Weinan E. Deep learning approximation for stochastic control problems. *arXiv preprint arXiv:1611.07422*, 2016. Cited on page 9.

Carsten Hartmann and Christof Schütte. Efficient rare event simulation by optimal nonequilibrium forcing. *Journal of Statistical Mechanics: Theory and Experiment*, 2012(11):P11004, 2012. Cited on page 9.

Jack Hessel, Ari Holtzman, Maxwell Forbes, Ronan Le Bras, and Yejin Choi. Clipscore: A reference-free evaluation metric for image captioning. *arXiv preprint arXiv:2104.08718*, 2021. Cited on pages 2 and 14.

Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. *arXiv preprint arXiv:2207.12598*, 2022. Cited on pages 2, 15, and 25.

Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, volume 33. Curran Associates, Inc., 2020. Cited on pages 2, 3, 4, and 7.

Yujia Huang, Adishree Ghatare, Yuanzhe Liu, Ziniu Hu, Qinsheng Zhang, Chandramouli S Sastry, Siddharth Gururani, Sageev Oore, and Yisong Yue. Symbolic music generation with non-differentiable rule guided diffusion. *arXiv preprint arXiv:2402.14285*, 2024. Cited on page 12.

Gabriel Ilharco, Mitchell Wortsman, Ross Wightman, Cade Gordon, Nicholas Carlini, Rohan Taori, Achal Dave, Vaishaal Shankar, Hongseok Namkoong, John Miller, Hannaneh Hajishirzi, Ali Farhadi, and Ludwig Schmidt. Openclip, July 2021. Cited on page 54.

H J Kappen. Path integrals and symmetry breaking for optimal control theory. *Journal of Statistical Mechanics: Theory and Experiment*, 2005(11), nov 2005. Cited on pages 5 and 9.

Hilbert J Kappen, Vicenç Gómez, and Manfred Opper. Optimal control as a graphical model inference problem. *Machine learning*, 87(2):159–182, 2012. Cited on page 9.

Diederik P Kingma, Tim Salimans, Ben Poole, and Jonathan Ho. On density estimation with diffusion models. In A. Beygelzimer, Y. Dauphin, P. Liang, and J. Wortman Vaughan, editors, *Advances in Neural Information Processing Systems*, 2021. Cited on page 2.

Yuval Kirstain, Adam Polyak, Uriel Singer, Shahbuland Matiana, Joe Penna, and Omer Levy. Pick-a-pic: An open dataset of user preferences for text-to-image generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. Cited on pages 2, 14, and 54.

Matthew Le, Apoorv Vyas, Bowen Shi, Brian Karrer, Leda Sari, Rashel Moritz, Mary Williamson, Vimal Manohar, Yossi Adi, Jay Mahadeokar, et al. Voicebox: Text-guided multilingual universal speech generation at scale. *Advances in neural information processing systems*, 36, 2024. Cited on page 2.

Dongzhuo Li. Differentiable gaussianization layers for inverse problems regularized by deep generative models. *arXiv preprint arXiv:2112.03860*, 2021. Cited on page 12.

Xuechen Li, Ting-Kam Leonard Wong, Ricky T. Q. Chen, and David Duvenaud. Scalable gradients for stochastic differential equations. In *International Conference on Artificial Intelligence and Statistics*, pages 3870–3882. PMLR, 2020. Cited on pages 9 and 12.

Yaron Lipman, Ricky T. Q. Chen, Heli Ben-Hamu, Maximilian Nickel, and Matthew Le. Flow matching for generative modeling. In *The Eleventh International Conference on Learning Representations*, 2023. Cited on pages 2, 3, 12, 13, and 36.

Guan-Horng Liu, Yaron Lipman, Maximilian Nickel, Brian Karrer, Evangelos Theodorou, and Ricky T. Q. Chen. Generalized schrödinger bridge matching. In *The Twelfth International Conference on Learning Representations*, 2024. Cited on page 12.

Qiang Liu. Rectified flow: A marginal preserving approach to optimal transport. *arXiv preprint arXiv:2209.14577*, 2022. Cited on page 3.

Xingchao Liu, Chengyue Gong, and qiang liu. Flow straight and fast: Learning to generate and transfer data with rectified flow. In *The Eleventh International Conference on Learning Representations*, 2023. Cited on pages 2 and 3.

Jonathan Long, Evan Shelhamer, and Trevor Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 3431–3440, 2015. Cited on page 13.

Nikolay Malkin, Moksh Jain, Emmanuel Bengio, Chen Sun, and Yoshua Bengio. Trajectory balance: Improved credit assignment in gflownets. *arXiv preprint arXiv:2201.13259*, 2023. Cited on page 13.

Dimitra Maoutsa, Sebastian Reich, and Manfred Opper. Interacting particle solutions of fokker–planck equations through gradient–log–density estimation. *Entropy*, 22(8):802, 2020. Cited on page 3.

Shakir Mohamed, Mihaela Rosca, Michael Figurnov, and Andriy Mnih. Monte carlo gradient estimation in machine learning. *Journal of Machine Learning Research*, 21(132):1–62, 2020. Cited on page 9.

Alexander Mordvintsev, Christopher Olah, and Mike Tyka. Inceptionism: Going deeper into neural networks. *Google research blog*, 20(14):5, 2015. Cited on page 2.

Nikolas Nüsken and Lorenz Richter. Solving high-dimensional Hamilton–Jacobi–Bellman pdes using neural networks: perspectives from the theory of controlled diffusions and measures on path space. *Partial differential equations and applications*, 2:1–48, 2021. Cited on page 13.

Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, Pamela Mishkin, Chong Zhang, Sandhini Agarwal, Katarina Slama, Alex Ray, John Schulman, Jacob Hilton, Fraser Kelton, Luke Miller, Maddie Simens, Amanda Askell, Peter Welinder, Paul F Christiano, Jan Leike, and Ryan Lowe. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems*, volume 35, pages 27730–27744. Curran Associates, Inc., 2022. Cited on pages 2, 6, and 12.

Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. Pytorch: An imperative style, high-performance deep learning library. *Advances in neural information processing systems*, 32, 2019. Cited on page 9.

Ashwini Pokle, Matthew J Muckley, Ricky T. Q. Chen, and Brian Karrer. Training-free linear image inversion via flows. *arXiv preprint arXiv:2310.04432*, 2023. Cited on page 12.

L.S. Pontryagin. *The Mathematical Theory of Optimal Processes*. Interscience Publishers, 1962. Cited on pages 8 and 9.

Aram-Alexandre Pooladian, Carles Domingo-Enrich, Ricky T. Q. Chen, and Brandon Amos. Neural optimal transport with lagrangian costs. *arXiv preprint arXiv:2406.00288*, 2024. Cited on page 12.

Rafael Rafailov, Archit Sharma, Eric Mitchell, Christopher D Manning, Stefano Ermon, and Chelsea Finn. Direct preference optimization: Your language model is secretly a reward model. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. Cited on pages 6 and 12.

Konrad Rawlik, Marc Toussaint, and Sethu Vijayakumar. On stochastic optimal control and reinforcement learning by approximate inference. In *Twenty-Third International Joint Conference on Artificial Intelligence*, 2013. Cited on pages 6 and 9.

Lorenz Richter and Julius Berner. Improved sampling via learned diffusions. In *The Twelfth International Conference on Learning Representations*, 2024. Cited on page 12.

Lorenz Richter, Ayman Boustati, Nikolas Nüsken, Francisco Ruiz, and Omer Deniz Akyildiz. VarGrad: A low-variance gradient estimator for variational inference. *Advances in Neural Information Processing Systems*, 33, 2020. Cited on page 13.

Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022. Cited on pages 2 and 13.

Litu Rout, Yujia Chen, Nataniel Ruiz, Abhishek Kumar, Constantine Caramanis, Sanjay Shakkottai, and Wen-Sheng Chu. Rb-modulation: Training-free personalization of diffusion models using stochastic optimal control. *arXiv preprint arXiv:2405.17401*, 2024. Cited on page 12.

Reuven Y Rubinstein and Dirk P Kroese. *The cross-entropy method: a unified approach to combinatorial optimization, Monte-Carlo simulation and machine learning*. Springer Science & Business Media, 2013. Cited on page 9.

Christoph Schuhmann and Romain Beaumont. Laion-aesthetics, 2022. Cited on page 2.

S.P. Sethi. *Optimal Control Theory: Applications to Management Science and Economics*. Springer International Publishing, 2018. Cited on page 5.

Uriel Singer, Adam Polyak, Thomas Hayes, Xi Yin, Jie An, Songyang Zhang, Qiyuan Hu, Harry Yang, Oron Ashual, Oran Gafni, et al. Make-a-video: Text-to-video generation without text-video data. *arXiv preprint arXiv:2209.14792*, 2022. Cited on page 2.

Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021a. Cited on pages 3, 7, and 31.

Jiaming Song, Arash Vahdat, Morteza Mardani, and Jan Kautz. Pseudoinverse-guided diffusion models for inverse problems. In *International Conference on Learning Representations*, 2023. Cited on page 12.

Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *arXiv preprint arXiv:1907.05600*, 2019. Cited on page 2.

Yang Song, Jascha Sohl-Dickstein, Diederik P. Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. In *International Conference on Learning Representations (ICLR 2021)*, 2021b. Cited on pages 2 and 3.

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel Ziegler, Ryan Lowe, Chelsea Voss, Alec Radford, Dario Amodei, and Paul F Christiano. Learning to summarize with human feedback. In *Advances in Neural Information Processing Systems*, volume 33, pages 3008–3021. Curran Associates, Inc., 2020. Cited on pages 2 and 12.

Hyung Ju Suh, Max Simchowitz, Kaiqing Zhang, and Russ Tedrake. Do differentiable simulators give better policy gradients? In *International Conference on Machine Learning*, pages 20668–20696. PMLR, 2022. Cited on page 9.

Wenpin Tang. Fine-tuning of diffusion models via stochastic control: entropy regularization and beyond. *arXiv preprint arXiv:2403.06279*, 2024. Cited on page 6.

Emanuel Todorov. Linearly-solvable markov decision problems. *Advances in neural information processing systems*, 19, 2006. Cited on page 5.

Belinda Tzen and Maxim Raginsky. Theoretical guarantees for sampling and inference in generative models with latent diffusions. *arXiv:1903.01608*, 2019. Cited on page 12.

Masatoshi Uehara, Yulai Zhao, Tommaso Biancalani, and Sergey Levine. Understanding reinforcement learning-based fine-tuning of diffusion models: A tutorial and review. *arXiv preprint arXiv:2407.13734*, 2024a. Cited on page 2.

Masatoshi Uehara, Yulai Zhao, Kevin Black, Ehsan Hajiramezanali, Gabriele Scalia, Nathaniel Lee Diamant, Alex M Tseng, Tommaso Biancalani, and Sergey Levine. Fine-tuning of continuous-time diffusion models as entropy-regularized control. *arXiv preprint arXiv:2402.15194*, 2024b. Cited on pages 2, 6, 12, and 37.

Francisco Vargas, Andrius Ovsianas, David Lopes Fernandes, Mark Girolami, Neil D Lawrence, and Nikolas Nüsken. Bayesian learning via neural schrödinger-föllmer flows. In *Fourth Symposium on Advances in Approximate Bayesian Inference*, 2022. Cited on page 12.

Francisco Vargas, Will Sussman Grathwohl, and Arnaud Doucet. Denoising diffusion samplers. In *The Eleventh International Conference on Learning Representations*, 2023. Cited on page 12.

Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural computation*, 23(7): 1661–1674, 2011. Cited on page 12.

Apoorv Vyas, Bowen Shi, Matthew Le, Andros Tjandra, Yi-Chiao Wu, Baishan Guo, Jiemin Zhang, Xinyue Zhang, Robert Adkins, William Ngan, et al. Audiobox: Unified audio generation with natural language prompts. *arXiv preprint arXiv:2312.15821*, 2023. Cited on page 2.

Bram Wallace, Meihua Dang, Rafael Rafailov, Linqi Zhou, Aaron Lou, Senthil Purushwalkam, Stefano Ermon, Caiming Xiong, Shafiq Joty, and Nikhil Naik. Diffusion model alignment using direct preference optimization. *arXiv preprint arXiv:2311.12908*, 2023a. Cited on pages 2, 14, 22, and 52.

Bram Wallace, Akash Gokul, Stefano Ermon, and Nikhil Naik. End-to-end diffusion latent optimization improves classifier guidance. *arXiv preprint arXiv:2303.13703*, 2023b. Cited on page 12.

Luhuan Wu, Brian Trippe, Christian Naesseth, David Blei, and John P Cunningham. Practical and asymptotically exact conditional sampling in diffusion models. In *Advances in Neural Information Processing Systems*, volume 36, pages 31372–31403. Curran Associates, Inc., 2023a. Cited on pages 12 and 13.

Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. *arXiv preprint arXiv:2306.09341*, 2023b. Cited on pages 14 and 54.

Xiaoshi Wu, Yiming Hao, Keqiang Sun, Yixiong Chen, Feng Zhu, Rui Zhao, and Hongsheng Li. Human preference score v2: A solid benchmark for evaluating human preferences of text-to-image synthesis. *arXiv preprint arXiv:2306.09341*, 2023c. Cited on page 2.

Jiazheng Xu, Xiao Liu, Yuchen Wu, Yuxuan Tong, Qinkai Li, Ming Ding, Jie Tang, and Yuxiao Dong. Imagereward: Learning and evaluating human preferences for text-to-image generation. In *Thirty-seventh Conference on Neural Information Processing Systems*, 2023. Cited on pages 2, 12, 13, 14, 22, and 51.

Dinghuai Zhang, Yizhe Zhang, Jiatao Gu, Ruixiang Zhang, Josh Susskind, Navdeep Jaitly, and Shuangfei Zhai. Improving gflownets for text-to-image diffusion alignment. *arXiv preprint arXiv:2406.00633*, 2024. Cited on page 12.

Qinsheng Zhang and Yongxin Chen. Path integral sampler: A stochastic control approach for sampling. In *International Conference on Learning Representations*, 2022. Cited on page 12.

Wei Zhang, Han Wang, Carsten Hartmann, Marcus Weber, and Christof Schütte. Applications of the cross-entropy method to importance sampling and optimal control of diffusions. *SIAM Journal on Scientific Computing*, 36(6): A2654–A2672, 2014. Cited on page 9.

Qinqing Zheng, Matt Le, Neta Shaul, Yaron Lipman, Aditya Grover, and Ricky T. Q. Chen. Guided flows for generative modeling and decision making. *arXiv preprint arXiv:2311.13443*, 2023. Cited on pages 2 and 15.

Brian D Ziebart, Andrew L Maas, J Andrew Bagnell, Anind K Dey, et al. Maximum entropy inverse reinforcement learning. In *Aaai*, volume 8, pages 1433–1438. Chicago, IL, USA, 2008. Cited on pages 5 and 37.

Daniel M. Ziegler, Nisan Stiennon, Jeffrey Wu, Tom B. Brown, Alec Radford, Dario Amodei, Paul Christiano, and Geoffrey Irving. Fine-tuning language models from human preferences. *arXiv preprint arXiv:1909.08593*, 2020. Cited on pages 2 and 12.

Appendix

Contents

where we used that $Y_1 = (x - \beta_t Y_0)/\alpha_t$. Also, we can write the score as follows

$$\mathfrak{s}(x,t) := \nabla \log p_t(x) = \frac{\nabla p_t(x)}{p_t(x)} = \frac{\nabla \mathbb{E}[p_{t|1}(x|Y_1)]}{p_t(x)} = \frac{\mathbb{E}[\nabla p_{t|1}(x|Y_1)]}{p_t(x)} = \frac{\mathbb{E}[p_{t|1}(x|Y_1)\nabla \log p_{t|1}(x|Y_1)]}{p_t(x)}, \tag{104}$$

where

$$p_{t|1}(x|Y_1) = \frac{\exp(-\|x-\alpha_t Y_1\|^2/(2\beta_t^2))}{(2\pi\beta_t^2)^{d/2}} \implies \nabla \log \vec{p}_{t|1}(x|Y_1) = -\frac{x-\alpha_t Y_1}{\beta_t^2} \tag{105}$$

Plugging this back into the right-hand side of (104), we obtain

$$\begin{aligned} \mathfrak{s}(x,t) &= -\frac{\mathbb{E}[p_{t|1}(x|Y_1)\frac{x-\alpha_t Y_1}{\beta_t^2}]}{p_t(x)} = -\frac{\int \vec{p}_{t|1}(x|Y_1)p_1(Y_1)\frac{x-\alpha_t Y_1}{\beta_t^2}\, dY_1}{\vec{p}_t(x)} \\ &= -\int p_{1|t}(Y_1|x)\frac{x-\alpha_t Y_1}{\beta_t^2}\, dY_1 = -\mathbb{E}[\frac{x-\alpha_t Y_1}{\beta_t^2}|x = \alpha_t Y_1 + \beta_t Y_0] = -\frac{\mathbb{E}[Y_0|x=\alpha_t Y_1+\beta_t Y_0]}{\beta_t} \end{aligned} \tag{106}$$

The last equality holds because $(x - \alpha_t Y_1)/\beta_t = Y_0$. Putting together (103) and (106), we obtain that

$$v(x,t) = \frac{\dot{\alpha}_t}{\alpha_t}x + \beta_t(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t - \dot{\beta}_t)\mathfrak{s}(x,t) \iff \mathfrak{s}(x,t) = \frac{1}{\beta_t(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t - \dot{\beta}_t)}\big(v(x,t) - \frac{\dot{\alpha}_t}{\alpha_t}x\big) \tag{107}$$

Thus, the ODE (3) can be rewritten like this:

$$\frac{\mathrm{d}X_t}{\mathrm{d}t} = \frac{\dot{\alpha}_t}{\alpha_t}X_t + \beta_t(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t - \dot{\beta}_t)\mathfrak{s}(X_t,t), \qquad X_0 \sim p_0. \tag{108}$$

To allow for an arbitrary diffusion coefficient, we need to add a correction term to the drift:

$$\mathrm{d}X_t = \big(\frac{\dot{\alpha}_t}{\alpha_t}X_t + \big(\frac{\sigma(t)^2}{2} + \beta_t(\frac{\dot{\alpha}_t}{\alpha_t}\beta_t - \dot{\beta}_t)\big)\mathfrak{s}(X_t,t)\big)\mathrm{d}t + \sigma(t)\mathrm{d}B_t, \qquad X_0 \sim p_0. \tag{109}$$

This can be easily shown by writing down the Fokker-Planck equations for (108) and (109), and observing that they are the same up to a cancellation of terms. Finally, if we plug the right-hand side of (107) into (109), we obtain the SDE for Flow Matching with arbitrary noise schedule (equation (4)).

C Stochastic optimal control as maximum entropy RL in continuous space and time

In this section, we bridge KL-regularized (or MaxEnt) reinforcement learning and stochastic optimal control. We show that when the action space is Euclidean and the transition probabilities are conditional Gaussians, taking the limit in which the stepsize goes to zero on the KL-regularized RL problem gives rise to the SOC problem. A consequence of this connection is that all algorithms for KL-regularized RL admit an analog for diffusion fine-tuning. This is not novel, but it may be useful for researchers that are familiar with RL fine-tuning formulations.

Appendix C.4 is providing a more direct, rigorous, continuous-time connection between SOC and MaxEnt RL, as it shows that the expected control cost is equal to the KL divergence between the distributions over trajectories, conditioned on the starting points (see equation (18)).

C.1 Maximum entropy RL

Several diffusion fine-tuning methods (Black et al., 2024; Uehara et al., 2024b) are based on KL-regularized RL, also known as maximum entropy RL, which we review in the following. In the classical reinforcement learning (RL) setting, we have an agent that, starting from state $s_0 \sim p_0$, iteratively observes a state s_k, takes an action a_k according to a policy $\pi(a_k; s_k, k)$ which leads to a new state s_{k+1} according to a fixed transition probability $p(s_{k+1}|a_k, s_k)$, and obtains rewards $r_k(s_k, a_k)$. This can be summarized into a trajectory $\tau = ((s_k, a_k))_{k=0}^K$. The goal is to optimize the policy π in order to maximize the expected total reward, i.e. $\max_\pi \mathbb{E}_{\tau\sim\pi,p}[\sum_{k=0}^K r_k(s_k, a_k)]$.

Maximum entropy RL (MaxEnt RL; Ziebart et al. (2008)) amounts to adding the entropy $H(\pi)$ of the policy $\pi(\cdot; s_k, k)$ to the reward for each step k, in order to encourage exploration and improve robustness to changes

in the environment: $\max_\pi \mathbb{E}_{\tau\sim\pi,p}[\sum_{k=0}^{K} r_k(s_k,a_k) + \sum_{k=0}^{K-1} H(\pi(\cdot;s_k,k))]$ [8]. As a generalization, one can regularize using the negative KL divergence between $\pi(\cdot;s_k,k)$ and a base policy $\pi_{\text{base}}(\cdot;s_k,k)$:

$$\max_\pi \mathbb{E}_{\tau\sim\pi,p}[\textstyle\sum_{k=0}^{K} r_k(s_k,a_k) - \sum_{k=0}^{K-1} \mathrm{KL}(\pi(\cdot;s_k,k)||\pi_{\text{base}}(\cdot;s_k,k))], \tag{110}$$

which prevents the learned policy to deviate too much from the base policy. Each policy π induces a distribution $q(\tau)$ over trajectories τ, and the MaxEnt RL problem (110) can be expressed solely in terms of such distributions (Lemma 3 in Appendix C.3):

$$\max_q \mathbb{E}_{\tau\sim q}[\textstyle\sum_{k=0}^{K} r_k(s_k,a_k)] - \mathrm{KL}(q||q^{\text{base}}), \tag{111}$$

where q^{base} is the distribution induced by the base policy π_{base}, and the maximization is over all distributions q such that their marginal for s_0 is p_0. We can further recast this problem as (Lemma 4 in Appendix C.3):

$$\min_q \mathrm{KL}(q||q^*), \qquad \text{where } q^*(\tau) := q^{\text{base}}(\tau)\exp\big(\textstyle\sum_{k=0}^{K} r_k(s_k,a_k) - \mathcal{V}(s_0,0)\big), \tag{112}$$

where

$$\begin{aligned}\mathcal{V}(s_k,k) &:= \log\big(\mathbb{E}_{\tau\sim\pi_{\text{base}},p}[\exp\big(\textstyle\sum_{k'=k}^{K} r_{k'}(s_{k'},a_{k'})\big)|s_k]\big)\\ &= \max_\pi \mathbb{E}_{\tau\sim\pi,p}\big[\textstyle\sum_{k'=k}^{K} r_{k'}(s_{k'},a_{k'}) - \sum_{k'=k}^{K-1} \mathrm{KL}(\pi(\cdot;s_{k'},k')||\pi_{\text{base}}(\cdot;s_{k'},k'))|s_k\big]\end{aligned} \tag{113}$$

is the value function. Problem (112) directly implies that the distribution induced by the optimal policy π^* is the tilted distribution q^* (which has initial marginal p_0).

C.2 From maximum entropy RL to stochastic optimal control

The following well-known result, which we prove in Appendix C.3, shows that in a natural sense, the continuous-time continuous-space version of MaxEnt RL is the SOC framework introduced in Section 4.1. In particular, when states and actions are vectors in $\mathbb{R}^d$, policies are specified by a vector field u (the control), and transition probabilities are conditional Gaussians, the MaxEnt RL problem becomes an SOC problem when the number of timesteps grows to infinity.

Proposition 5. *Suppose that*

(i) The state space and the action space are $\mathbb{R}^d$,

(ii) Policies π are specified as $\pi(a_k;s_k,k) = \delta(a_k - u(s_k,kh))$, where $u:\mathbb{R}^d\times[0,T]\to\mathbb{R}^d$ is a vector field, and δ denotes the Dirac delta,

(iii) Transition probabilities are conditional Gaussian densities: $p(s_{k+1}|a_k,s_k) = N(s_k + h(b(s_k,kh) + \sigma(kh)a_k), h\sigma(kh)\sigma(kh)^\top)$, where $h = T/K$ is the stepsize, and b and σ are defined as in Section 4.1.

Then, in the limit in which the number of steps K grows to infinity, the problem (110) is equivalent to the SOC problem (12)-(13), identifying

- *the sequence of states $(s_k)_{k=0}^k$ with the trajectory $\boldsymbol{X}^u = (X_t^u)_{t\in[0,1]}$,*
- *the running reward $\sum_{k=0}^{K-1} r_k(s_k,a_k)$ with the negative running cost $-\int_0^T f(X_t^u,t)\,\mathrm{d}t$,*
- *the terminal reward $r_K(s_K,a_K)$ with the negative terminal cost $-g(X_T^u)$,*
- *the KL regularization $\mathbb{E}_{\tau\sim\pi,p}[\sum_{k=0}^{K-1}\mathrm{KL}(\pi(\cdot;s_k,k)||\pi_{\text{base}}(\cdot;s_k,k))]$ with $\frac{1}{2}$ times the expected L^2 norm of the control $\frac{1}{2}\mathbb{E}\big[\int_0^T \|u(X_t^u,t)\|^2\,\mathrm{d}t\big]$,*
- *and the value function $\mathcal{V}(s_k,k)$ defined in (113) with the negative value function $-V(x,t)$ defined in Section 4.1.*

A first consequence of this result is that every loss function designed for generic MaxEnt RL problems has a corresponding loss function for SOC problems. The geometric structure of the latter allows for additional

[8] The entropy terms are usually multiplied by a factor to tune their magnitude, but one can equivalently rescale the rewards, which is why we do not add any factor.

losses that do not have an analog in the classical MaxEnt RL setting; in particular, we can differentiate the state and terminal costs.

A second consequence of Proposition 5 is that the characterization (112) can be translated to the SOC setting. The analogs of the distributions q^*, q^{base} induced by the optimal policy π^* and the base policy π^{base} are the distributions p^*, p^{base} induced by the optimal control u^* and the null control. For an arbitrary trajectory $\boldsymbol{X} = (X_t)_{t\in[0,T]}$, the relation between $\mathbb{P}^*$ and $\mathbb{P}^{\text{base}}$ is given by

$$\frac{\mathrm{d}\mathbb{P}^*}{\mathrm{d}\mathbb{P}^{\text{base}}}(\boldsymbol{X}) = \exp(-\int_0^T f(X_t,t)\,\mathrm{d}t - g(X_T) + V(X_0,0)) \tag{114}$$

where V is the value function as defined in Section 4.1. Note that this matches the statement in (22).

C.3 Proof of Proposition 5: from MaxEnt RL to SOC

Since the transition $p(s_{k+1}|a_k,s_k)$ is fixed, for each π we can define

$$\tilde{\pi}(a_k,s_{k+1};s_k,k) = \pi(a_k;s_k,k)p(s_{k+1}|a_k,s_k) \text{ and } \tilde{\pi}_{\text{base}}(a_k,s_{k+1};s_k,k) = \pi_{\text{base}}(a_k;s_k,k)p(s_{k+1}|a_k,s_k), \tag{115}$$

and reexpress (110) as (see Lemma 3)

$$\min_{\tilde{\pi}} \mathbb{E}_{\tau\sim\tilde{\pi}}[\textstyle\sum_{k=0}^K r_k(s_k,a_k) - \sum_{k=0}^{K-1} \mathrm{KL}(\tilde{\pi}(\cdot,\cdot;s_k,k)||\tilde{\pi}_{\text{base}}(\cdot,\cdot;s_k,k))]. \tag{116}$$

Using the hypothesis of the proposition, we can write

$$\begin{aligned}\tilde{\pi}(a_k,s_{k+1};s_k,k) &= \delta(a_k - u(s_k,k\eta))N(s_k + \eta(b(s_k,k\eta) + \sigma(k\eta)a_k), \eta\sigma(k\eta)\sigma(k\eta)^\top) \\ &= \delta(a_k - u(s_k,k\eta))\tilde{\pi}(s_{k+1};s_k,k),\end{aligned} \tag{117}$$

where $\tilde{\pi}(s_{k+1};s_k,k) = N(s_k + \eta(b(s_k,k\eta) + \sigma(k\eta)u(s_k,k\eta)), \eta\sigma(k\eta)\sigma(k\eta)^\top)$ is the state transition kernel. We set the base policy as $\pi_{\text{base}}(a_k;s_k,k) = \delta(a_k)$, and we obtain analogously that $\tilde{\pi}(a_k,s_{k+1};s_k,k) = \delta(a_k)\tilde{\pi}_{\text{base}}(s_{k+1};s_k,k)$ with $\tilde{\pi}_{\text{base}}(s_{k+1};s_k,k) = N(s_k + \eta b(s_k,k\eta), \eta\sigma(k\eta)\sigma(k\eta)^\top)$. Now, if we take K large, the trajectory $(s_k)_{k=0}^K$ generated by $\tilde{\pi}$ can be regarded as the Euler-Maruyama discretization of a solution X^u of the controlled SDE (13), while the trajectory generated by $\tilde{\pi}_{\text{base}}$ is the discretization of the uncontrolled process X^0 obtained by setting $u = 0$. As a consequence

$$\begin{aligned}&\lim_{K\to\infty} \mathbb{E}_{\tau\sim\tilde{\pi}}[\textstyle\sum_{k=0}^{K-1} \mathrm{KL}(\tilde{\pi}(\cdot,\cdot;s_k,k)||\tilde{\pi}_{\text{base}}(\cdot,\cdot;s_k,k))] \\ &= \lim_{K\to\infty} \mathbb{E}_{\tau\sim\tilde{\pi}}[\textstyle\sum_{k=0}^{K-1} \mathrm{KL}(\tilde{\pi}(\cdot;s_k,k)||\tilde{\pi}_{\text{base}}(\cdot;s_k,k))] = \mathbb{E}_{X^u\sim\mathbb{P}^u}[\log \frac{\mathrm{d}\mathbb{P}^u}{\mathrm{d}\mathbb{P}^0}(X^u)],\end{aligned} \tag{118}$$

where $\mathbb{P}^u$ and $\mathbb{P}^0$ are the measures of the processes X^u and X^0, respectively. The Girsanov theorem (Theorem 2) implies that $\log \frac{\mathrm{d}\mathbb{P}^u}{\mathrm{d}\mathbb{P}^0}(X^u) = -\int_0^T \langle u(X_t^u,t), \mathrm{d}B_t\rangle - \frac{1}{2}\int_0^T \|u(X_t^u,t)\|^2\,\mathrm{d}t$, which implies that $\mathbb{E}_{X^u\sim\mathbb{P}^u}[\log \frac{\mathrm{d}\mathbb{P}^u}{\mathrm{d}\mathbb{P}^0}(X^u)] = -\frac{1}{2}\mathbb{E}_{X^u\sim\mathbb{P}^u}[\int_0^T \|u(X_t^u,t)\|^2\,\mathrm{d}t]$. Setting the rewards $r_k(a_k,s_k) = \eta f(s_k,k\eta)$ for $k \in \{0,\dots,K-1\}$ and $r_K(a_K,s_K) = \eta g(s_k)$, where f and g are as in Section 4.1, yields the following limiting object:

$$\lim_{K\to\infty} \mathbb{E}_{\tau\sim\tilde{\pi}}[\textstyle\sum_{k=0}^K r_k(s_k,a_k)] = \mathbb{E}_{X^u\sim\mathbb{P}^u}[\int_0^T f(X_t^u,t)\,dt + g(X_T^u)]. \tag{119}$$

Hence, the limit of the MaxEnt RL loss (116) is the SOC loss (12).

Lemma 3. *Let $\tilde{\pi}(a_k,s_{k+1};s_k,k)$ and $\tilde{\pi}_{\text{base}}(a_k,s_{k+1};s_k,k)$ be as defined in (115). $\mathrm{KL}(\tilde{\pi}(\cdot,\cdot;s_k,k)||\tilde{\pi}_{\text{base}}(\cdot,\cdot;s_k,k))]$ and $\mathrm{KL}(\pi(\cdot;s_k,k)||\pi_{\text{base}}(\cdot;s_k,k))]$ are equal. Moreover, if q, q^{base} denote the distributions over trajectories induced by π, π_{base}, we have that*

$$\mathrm{KL}(q||q^{\text{base}}) = \mathbb{E}[\textstyle\sum_{k=0}^{K-1} \mathrm{KL}(\pi(\cdot;s_k,k)||\pi_{\text{base}}(\cdot;s_k,k))]. \tag{120}$$

Proof. We have that

$$\begin{aligned}
&\mathrm{KL}(\tilde{\pi}(\cdot,\cdot;s_k,k)||\tilde{\pi}_{\text{base}}(\cdot,\cdot;s_k,k))] = \textstyle\sum_{a_k,s_{k+1}} \tilde{\pi}(a_k,s_{k+1};s_k,k)\log\frac{\tilde{\pi}(a_k,s_{k+1};s_k,k)}{\tilde{\pi}_{\text{base}}(a_k,s_{k+1};s_k,k)} \\
&= \textstyle\sum_{a_k,s_{k+1}} \pi(a_k;s_k,k)p(s_{k+1}|a_k,s_k)\log\frac{\pi(a_k;s_k,k)p(s_{k+1}|a_k,s_k)}{\pi_{\text{base}}(a_k;s_k,k)p(s_{k+1}|a_k,s_k)} \\
&= \textstyle\sum_{a_k,s_{k+1}} \pi(a_k;s_k,k)p(s_{k+1}|a_k,s_k)\log\frac{\pi(a_k;s_k,k)}{\pi_{\text{base}}(a_k;s_k,k)} \\
&= \textstyle\sum_{a_k} \pi(a_k;s_k,k)\big(\sum_{s_{k+1}} p(s_{k+1}|a_k,s_k)\big)\log\frac{\pi(a_k;s_k,k)}{\pi_{\text{base}}(a_k;s_k,k)} \\
&= \textstyle\sum_{a_k} \pi(a_k;s_k,k)\log\frac{\pi(a_k;s_k,k)}{\pi_{\text{base}}(a_k;s_k,k)} = \mathrm{KL}(\pi(\cdot;s_k,k)||\pi_{\text{base}}(\cdot;s_k,k))].
\end{aligned} \tag{121}$$

To prove (120), by construction we can write

$$q(\tau) = p_0(s_0)\textstyle\prod_{k=0}^{K-1}\tilde{\pi}(a_k,s_{k+1};s_k,k), \qquad q^{\text{base}}(\tau) = p_0(s_0)\prod_{k=0}^{K-1}\tilde{\pi}_{\text{base}}(a_k,s_{k+1};s_k,k), \tag{122}$$

which means that

$$\begin{aligned}
\mathrm{KL}(q||q^{\text{base}}) &= \mathbb{E}_{\tau\sim q}[\log\tfrac{q(\tau)}{q^{\text{base}}(\tau)}] = \mathbb{E}_{\tau\sim q}[\textstyle\sum_{k=0}^{K-1}\log\frac{\tilde{\pi}(a_k,s_{k+1};s_k,k)}{\tilde{\pi}_{\text{base}}(a_k,s_{k+1};s_k,k)}] \\
&= \textstyle\sum_{k=0}^{K-1}\mathbb{E}_{\tau\sim q^{0:(k+1)}}[\log\frac{\tilde{\pi}(a_k,s_{k+1};s_k,k)}{\tilde{\pi}_{\text{base}}(a_k,s_{k+1};s_k,k)}] \\
&= \textstyle\sum_{k=0}^{K-1}\mathbb{E}_{\tau\sim q^{0:k}}[\sum_{a_k,s_{k+1}}\tilde{\pi}(a_k,s_{k+1};s_k,k)\log\frac{\tilde{\pi}(a_k,s_{k+1};s_k,k)}{\tilde{\pi}_{\text{base}}(a_k,s_{k+1};s_k,k)}] \\
&= \textstyle\sum_{k=0}^{K-1}\mathbb{E}_{\tau\sim q^{0:k}}[\mathrm{KL}(\tilde{\pi}(\cdot,\cdot;s_k,k)||\tilde{\pi}_{\text{base}}(\cdot,\cdot;s_k,k))] \\
&= \textstyle\sum_{k=0}^{K-1}\mathbb{E}_{\tau\sim q^{0:k}}[\mathrm{KL}(\pi(\cdot;s_k,k)||\pi_{\text{base}}(\cdot;s_k,k))] \\
&= \mathbb{E}_{\tau\sim q^{0:k}}[\textstyle\sum_{k=0}^{K-1}\mathrm{KL}(\pi(\cdot;s_k,k)||\pi_{\text{base}}(\cdot;s_k,k))]
\end{aligned} \tag{123}$$

Here, the notation $q^{0:k}$ denotes the trajectory q up to the state s_k. □

Lemma 4. *The distribution-based MaxEnt RL formulation in (111) is equivalent to the the following problem:*

$$\min_q \mathrm{KL}(q||q^*), \qquad \textit{where } q^*(\tau) := \frac{q^{\text{base}}(\tau)\exp\big(\sum_{k=0}^K r_k(s_k,a_k)\big)}{\frac{1}{p_0(s_0)}\sum_{\{\tau'|s'_0=s_0\}} q^{\text{base}}(\tau')\exp\big(\sum_{k=0}^K r_k(s'_k,a'_k)\big)}, \tag{124}$$

where the minimization is over q with marginal p_0 at step zero. The optimum of the problem is q^*, which satisfies the marginal constraint. The following alternative characterization of q^* holds:

$$q^*(\tau) = q^{\text{base}}(\tau)\exp\big(\textstyle\sum_{k=0}^K r_k(s_k,a_k) - \mathcal{V}(s_0,0)\big), \tag{125}$$

$$\textit{where } \mathcal{V}(x,k) = \max_\pi \mathbb{E}_{\tau\sim\pi,p}\big[\textstyle\sum_{k'=k}^K r_{k'}(s_{k'},a_{k'}) - \sum_{k'=k}^{K-1}\mathrm{KL}(\pi(\cdot;s_{k'},k')||\pi_{\text{base}}(\cdot;s_{k'},k'))|s_k=x\big]. \tag{126}$$

Proof. Let us expand $\mathrm{KL}(q||q^*)$:

$$\begin{aligned}
\mathrm{KL}(q||q^*) &= \mathbb{E}_{\tau\sim q}\big[\log\tfrac{q(\tau)}{q^*(\tau)}\big] \\
&= \mathbb{E}_{\tau\sim q}\big[\log q(\tau) - \log q^{\text{base}}(\tau) - \textstyle\sum_{k=0}^K r_k(s_k,a_k) \\
&\qquad + \log\big(\tfrac{1}{p_0(s_0)}\textstyle\sum_{\{\tau'|s'_0=s_0\}} q^{\text{base}}(\tau')\exp\big(\sum_{k=0}^K r_k(s'_k,a'_k)\big)\big)\big] \\
&= \mathrm{KL}(q||q^{\text{base}}) - \mathbb{E}_{\tau\sim q}\big[\textstyle\sum_{k=0}^K r_k(s_k,a_k)\big] \\
&\qquad + \mathbb{E}_{s_0\sim p_0}\big[\log\big(\tfrac{1}{p_0(s_0)}\textstyle\sum_{\{\tau'|s'_0=s_0\}} q^{\text{base}}(\tau')\exp\big(\sum_{k=0}^K r_k(s'_k,a'_k)\big)\big)\big],
\end{aligned} \tag{127}$$

where the third equality holds because the marginal of q at step zero is p_0 by hypothesis. Since the third term in the right-hand side is independent of q, this proves the equivalence between (111) and (124).

Next, we prove that the marginal of q^* at step zero is p_0:

$$\textstyle\sum_{\{\tau|s_0=x\}} q^*(\tau) := \sum_{\{\tau|s_0=x\}} \frac{q^{\text{base}}(\tau)\exp\big(\sum_{k=0}^K r_k(s_k,a_k)\big)}{\frac{1}{p_0(x)}\sum_{\{\tau'|s'_0=x\}} q^{\text{base}}(\tau')\exp\big(\sum_{k=0}^K r_k(s'_k,a'_k)\big)} = p_0(x). \tag{128}$$

Now, for an arbitrary s_0, let q_{s_0}, $q^*_{s_0}$ be the distributions q, q^* conditioned on the initial state being s_0. We can write an analog to equation (127) for q_{s_0}, $q^*_{s_0}$:

$$\begin{aligned}\mathrm{KL}(q_{s_0}||q^*_{s_0}) &= \mathbb{E}_{\tau\sim q_{s_0}}\big[\log \tfrac{q_{s_0}(\tau)}{q^*_{s_0}(\tau)}\big] \\ &= \mathbb{E}_{\tau\sim q_{s_0}}\big[\log q_{s_0}(\tau) - \log q^{\text{base}}_{s_0}(\tau) - \textstyle\sum_{k=0}^K r_k(s_k,a_k) \\ &\qquad + \log\big(\tfrac{1}{p_0(s_0)}\textstyle\sum_{\{\tau'|s'_0=s_0\}} q^{\text{base}}_{s_0}(\tau')\exp\big(\textstyle\sum_{k=0}^K r_k(s'_k,a'_k)\big)\big)\big] \\ &= \mathrm{KL}(q_{s_0}||q^{\text{base}}_{s_0}) - \mathbb{E}_{\tau\sim q_{s_0}}\big[\textstyle\sum_{k=0}^K r_k(s_k,a_k)\big] \\ &\qquad + \log\big(\tfrac{1}{p_0(s_0)}\textstyle\sum_{\{\tau'|s'_0=s_0\}} q^{\text{base}}(\tau')\exp\big(\textstyle\sum_{k=0}^K r_k(s'_k,a'_k)\big)\big),\end{aligned} \tag{129}$$

Hence,

$$\begin{aligned}0 = \min_{q_{s_0}} \mathrm{KL}(q_{s_0}||q^*_{s_0}) = -\max_{q_{s_0}}\{\mathbb{E}_{\tau\sim q_{s_0}}\big[\textstyle\sum_{k=0}^K r_k(s_k,a_k)\big] - \mathrm{KL}(q_{s_0}||q^{\text{base}}_{s_0})\} \\ + \log\big(\tfrac{1}{p_0(s_0)}\textstyle\sum_{\{\tau'|s'_0=s_0\}} q^{\text{base}}(\tau')\exp\big(\textstyle\sum_{k=0}^K r_k(s'_k,a'_k)\big)\big).\end{aligned} \tag{130}$$

And applying (120) from (120), we obtain that

$$\begin{aligned}&\log\big(\tfrac{1}{p_0(s_0)}\textstyle\sum_{\{\tau'|s'_0=s_0\}} q^{\text{base}}(\tau')\exp\big(\textstyle\sum_{k=0}^K r_k(s'_k,a'_k)\big)\big) \\ &= \max_\pi \mathbb{E}_{\tau\sim\pi,p}\big[\textstyle\sum_{k=0}^K r_k(s_k,a_k) - \sum_{k=0}^{K-1}\mathrm{KL}(\pi(\cdot;s_k,k)||\pi_{\text{base}}(\cdot;s_k,k))|s_0\big] = \mathcal{V}(s_0,0),\end{aligned} \tag{131}$$

which concludes the proof. □

C.4 Proof of equation (18): the control cost is a KL regularizer

Theorem 2 (Girsanov theorem for SDEs)**.** *If the two SDEs*

$$\mathrm{d}X_t = b_1(X_t,t)\,\mathrm{d}t + \sigma(X_t,t)\,\mathrm{d}B_t, \qquad X_0 = x_{\text{init}} \tag{132}$$

$$dY_t = (b_1(Y_t,t) + b_2(Y_t,t))\,\mathrm{d}t + \sigma(Y_t,t)\,\mathrm{d}B_t, \qquad Y_0 = x_{\text{init}} \tag{133}$$

admit unique strong solutions on $[0,T]$, then for any bounded continuous functional Φ on $C([0,T])$, we have that

$$\begin{aligned}\mathbb{E}[\Phi(\boldsymbol{X})] &= \mathbb{E}\big[\Phi(\boldsymbol{Y})\exp\big(-\textstyle\int_0^T \sigma(Y_t,t)^{-1}b_2(Y_t,t)\,\mathrm{d}B_t - \frac{1}{2}\int_0^T \|\sigma(Y_t,t)^{-1}b_2(Y_t,t)\|^2\,\mathrm{d}t\big)\big] \\ &= \mathbb{E}\big[\Phi(\boldsymbol{Y})\exp\big(-\textstyle\int_0^T \sigma(Y_t,t)^{-1}b_2(Y_t,t)\,d\tilde{B}_t + \frac{1}{2}\int_0^T \|\sigma(Y_t,t)^{-1}b_2(Y_t,t)\|^2\,\mathrm{d}t\big)\big],\end{aligned} \tag{134}$$

where $\tilde{B}_t = B_t + \int_0^t \sigma(Y_s,s)^{-1}b_2(Y_s,s)\,\mathrm{d}s$. More generally, b_1 and b_2 can be random processes that are adapted to filtration of $\boldsymbol{B}$.

Consider the SDEs

$$\mathrm{d}X_t = b(X_t,t)\,\mathrm{d}t + \sigma(t)\mathrm{d}B_t, \qquad X_0 = x_0, \tag{135}$$

$$\mathrm{d}X^u_t = \big(b(X^u_t,t) + \sigma(t)u(X^u_t,t)\big)\,\mathrm{d}t + \sigma(t)\mathrm{d}B_t, \qquad X^u_0 = x_0. \tag{136}$$

If we let $\mathbb{P}|_{x_0}$, $\mathbb{P}^u|_{x_0}$ be the probability measures of the solutions of (135) and (136), Theorem 2 implies that

$$\log\tfrac{\mathrm{d}\mathbb{P}|_{x_0}}{\mathrm{d}\mathbb{P}^u|_{x_0}}(\boldsymbol{X}^u) = -\textstyle\int_0^1 u(X^u_t,t)\,\mathrm{d}B_t - \frac{1}{2}\int_0^1 \|u(X^u_t,t)\|^2\,\mathrm{d}t. \tag{137}$$

Hence,

$$\begin{aligned}D_{\mathrm{KL}}\big(\mathbb{P}^u|_{x_0}\,\big\|\,\mathbb{P}|_{x_0}\big) &= \mathbb{E}\big[\log\tfrac{\mathrm{d}\mathbb{P}^u|_{x_0}}{\mathrm{d}\mathbb{P}|_{x_0}}(\boldsymbol{X}^u)|X^u_0 = x_0\big] = -\mathbb{E}\big[\log\tfrac{\mathrm{d}\mathbb{P}|_{x_0}}{\mathrm{d}\mathbb{P}^u|_{x_0}}(\boldsymbol{X}^u)|X^u_0 = x_0\big] \\ &= \mathbb{E}\big[\textstyle\int_0^1 u(X^u_t,t)\,\mathrm{d}B_t + \frac{1}{2}\int_0^1 \|u(X^u_t,t)\|^2\,\mathrm{d}t|X^u_0 = x_0\big] = \mathbb{E}\big[\frac{1}{2}\int_0^1 \|u(X^u_t,t)\|^2\,\mathrm{d}t|X^u_0 = x_0\big],\end{aligned} \tag{138}$$

where we used that stochastic integrals are martingales.

D Proofs of Section 4.3: memoryless noise schedule and fine-tuning recipe

D.1 Proof of Proposition 1: the memoryless noise schedule

We consider the forward-backward SDEs (63)-(64) with arbitrary noise schedule. By Proposition 4, the trajectories $\vec{\boldsymbol{X}}$, $\boldsymbol{X}$ of these two processes are equally distributed up to a time flip, which also means that their marginals satisfy $\vec{p}_t = p_{1-t}$, for all $t \in [0,1]$. First, we develop an explicit expression for the score function $s(x,t) = \nabla \log p_t(x)$. By the properties of flow matching, we know that p_t is the distribution of the interpolation variable $\bar{X}_t = \beta_t \bar{X}_0 + \alpha_t \bar{X}_1$, where $\bar{X}_0 \sim N(0, I), \bar{X}_1 \sim p^{\text{data}}$ are independent. Thus, $\frac{\bar{X}_t - \alpha_t \bar{X}_1}{\beta_t} \sim N(0, \mathrm{I})$, which means that we can express the density p_t as

$$p_t(x) = \int_{\mathbb{R}^d} \frac{\exp\left(-\frac{\|x-\alpha_t y\|^2}{2\beta_t^2}\right)}{(2\pi\beta_t^2)^{d/2}} p^{\text{data}}(y)\,\mathrm{d}y. \tag{139}$$

Thus,

$$s(x,t) = \nabla \log p_t(x) = -\frac{x}{\beta_t^2} + \frac{\alpha_t}{\beta_t^2} \frac{\int_{\mathbb{R}^d} y \exp\left(-\frac{\|x-\alpha_t y\|^2}{2\beta_t^2}\right) p^{\text{data}}(y)\,\mathrm{d}y}{\int_{\mathbb{R}^d} \exp\left(-\frac{\|x-\alpha_t y\|^2}{2\beta_t^2}\right) p^{\text{data}}(y)\,\mathrm{d}y} := -\frac{x - \alpha_t \xi_t(x)}{\beta_t^2}, \tag{140}$$

where we defined

$$\xi_t(x) = \frac{\int_{\mathbb{R}^d} y \exp\left(-\frac{\|x-\alpha_t y\|^2}{2\beta_t^2}\right) p^{\text{data}}(y)\,\mathrm{d}y}{\int_{\mathbb{R}^d} \exp\left(-\frac{\|x-\alpha_t y\|^2}{2\beta_t^2}\right) p^{\text{data}}(y)\,\mathrm{d}y}. \tag{141}$$

Hence, we can rewrite the forward SDE (63) as

$$\mathrm{d}\vec{X}_t = \Big(-\kappa_{1-t}\vec{X}_t - \Big(\frac{\sigma(1-t)^2}{2} - \eta_{1-t}\Big) \frac{\vec{X}_t - \alpha_{1-t}\xi_{1-t}(\vec{X}_t)}{\beta_{1-t}^2} \Big)\,\mathrm{d}t + \sigma(1-t)\,\mathrm{d}B_t, \qquad \vec{X}_0 \sim p_{\text{data}} \tag{142}$$

Hence, if we substitute $\kappa_{1-t} \leftarrow \kappa_{1-t} + \frac{\sigma(1-t)^2 - 2\eta_{1-t}}{2\beta_{1-t}^2}$, $\xi_{1-t} \leftarrow \frac{\alpha_{1-t}(\sigma(1-t)^2 - 2\eta_{1-t})}{2\beta_{1-t}^2}\xi_{1-t}(\vec{X}_t)$ (where we ignore the dependency on $\vec{X}_t$), $\sqrt{2\eta_{1-t}} \leftarrow \sigma(1-t)$, we can apply Lemma 2, which yields

$$\begin{aligned}
\vec{X}_t = \vec{X}_0 \exp\Big(-\int_0^t \Big(\kappa_{1-s} + \frac{\sigma(1-s)^2 - 2\eta_{1-s}}{2\beta_{1-s}^2}\Big)\,\mathrm{d}s\Big) \\
+ \int_0^t \exp\Big(-\int_{t'}^t \Big(\kappa_{1-s} + \frac{\sigma(1-s)^2 - 2\eta_{1-s}}{2\beta_{1-s}^2}\Big)\,\mathrm{d}s\Big) \frac{\alpha_{1-t'}(\sigma(1-t')^2 - 2\eta_{1-t'})}{2\beta_{1-t'}^2} \xi_{1-t'}(\vec{X}_{t'})\,\mathrm{d}t' \\
+ \int_0^t \sigma(1-t') \exp\Big(-\int_{t'}^t \Big(\kappa_{1-s} + \frac{\sigma(1-s)^2 - 2\eta_{1-s}}{2\beta_{1-s}^2}\Big)\,\mathrm{d}s\Big)\,\mathrm{d}B_{t'}.
\end{aligned} \tag{143}$$

We simplify the recurring expression:

$$\kappa_{1-s} + \frac{\sigma(1-s)^2 - 2\eta_{1-s}}{2\beta_{1-s}^2} = \frac{\dot{\alpha}_{1-s}}{\alpha_{1-s}} + \frac{\sigma(1-s)^2 - 2\beta_{1-s}\left(\frac{\dot{\alpha}_{1-s}}{\alpha_{1-s}}\beta_{1-s} - \dot{\beta}_{1-s}\right)}{2\beta_{1-s}^2} = \frac{\sigma(1-s)^2}{2\beta_{1-s}^2} + \frac{\dot{\beta}_{1-s}}{\beta_{1-s}} \tag{144}$$

Thus,

$$\int_{t'}^t \Big(\kappa_{1-s} + \frac{\sigma(1-s)^2 - 2\eta_{1-s}}{2\beta_{1-s}^2}\Big)\,\mathrm{d}s = \int_{t'}^t \Big(\frac{\sigma(1-s)^2}{2\beta_{1-s}^2} - \partial_s \log \beta_{1-s}\Big)\,\mathrm{d}s = \int_{t'}^t \frac{\sigma(1-s)^2}{2\beta_{1-s}^2}\,\mathrm{d}s - \big(\log\beta_{1-t} - \log\beta_{1-t'}\big), \tag{145}$$

which means that

$$\exp\Big(-\int_{t'}^t \Big(\kappa_{1-s} + \frac{\sigma(1-s)^2 - 2\eta_{1-s}}{2\beta_{1-s}^2}\Big)\,\mathrm{d}s\Big) = \exp\Big(-\int_{t'}^t \frac{\sigma(1-s)^2}{2\beta_{1-s}^2}\,\mathrm{d}s\Big) \frac{\beta_{1-t}}{\beta_{1-t'}}, \tag{146}$$

$$\frac{\alpha_{1-t'}(\sigma(1-t')^2 - 2\eta_{1-t'})}{2\beta_{1-t'}^2}\xi_{1-t'}(\vec{X}_{t'}) = \alpha_{1-t'}\Big(\frac{\sigma(1-t')^2}{2\beta_{1-t'}^2} + \frac{\dot{\beta}_{1-t'}}{\beta_{1-t'}} - \frac{\dot{\alpha}_{1-t'}}{\alpha_{1-t'}}\Big)\xi_{1-t'}(\vec{X}_{t'}). \tag{147}$$

If we define $\chi(1-s)$ such that $\sigma^2(1-s) = 2\beta_{1-s}\big(\frac{\dot\alpha_{1-s}}{\alpha_{1-s}}\beta_{1-s} - \dot\beta_{1-s}\big) + \chi(1-s)$, we obtain that

$$\begin{aligned}
&\exp\big(-\textstyle\int_{t'}^t \frac{\sigma(1-s)^2}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\beta_{1-t}}{\beta_{1-t'}} = \exp\big(-\textstyle\int_{t'}^t \big(\frac{\dot\alpha_{1-s}}{\alpha_{1-s}} - \frac{\dot\beta_{1-s}}{\beta_{1-s}} + \frac{\chi(1-s)}{2\beta_{1-s}^2}\big)\,\mathrm{d}s\big)\frac{\beta_{1-t}}{\beta_{1-t'}} \\
&= \exp\big(\textstyle\int_{t'}^t \big(\partial_s \log\alpha_{1-s} - \partial_s\log\beta_{1-s} - \frac{\chi(1-s)}{2\beta_{1-s}^2}\big)\,\mathrm{d}s\big)\frac{\beta_{1-t}}{\beta_{1-t'}} = \exp\big(-\textstyle\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\alpha_{1-t}}{\alpha_{1-t'}},
\end{aligned} \tag{148}$$

$$\alpha_{1-t'}\big(\tfrac{\sigma(1-t')^2}{2\beta_{1-t'}^2} + \tfrac{\dot\beta_{1-t'}}{\beta_{1-t'}} - \tfrac{\dot\alpha_{1-t'}}{\alpha_{1-t'}}\big)\xi_{1-t'}(\vec X_{t'}) = \tfrac{\alpha_{1-t'}\chi(1-t')}{2\beta_{1-t'}^2}\xi_{1-t'}(\vec X_{t'}) \tag{149}$$

If we plug equations (148)-(149) into (146)-(147), and then those into (143), we obtain that

$$\begin{aligned}
\vec X_t = \vec X_0 \exp\big(-\textstyle\int_0^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\alpha_{1-t}}{\alpha_1} + \alpha_{1-t}\textstyle\int_0^t \exp\big(-\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\chi(1-t')}{2\beta_{1-t'}^2}\xi_{1-t'}(\vec X_{t'})\,\mathrm{d}t' \\
+ \textstyle\int_0^t \big(2\beta_{1-t'}\big(\frac{\dot\alpha_{1-t'}}{\alpha_{1-t'}}\beta_{1-t'} - \dot\beta_{1-t'}\big) + \chi(1-t')\big)\exp\big(-\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\alpha_{1-t}}{\alpha_{1-t'}}\,\mathrm{d}B_{t'}.
\end{aligned} \tag{150}$$

and if we take the limit $t \to 1^-$ and use that $\alpha_1 = 1$,

$$\begin{aligned}
\vec X_1 = \vec X_0\big(\lim_{t\to1^-}\exp\big(-\textstyle\int_0^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\alpha_{1-t}\big) + \lim_{t\to1^-}\alpha_{1-t}\textstyle\int_0^t \exp\big(-\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\chi(1-t')}{2\beta_{1-t'}^2}\xi_{1-t'}(\vec X_{t'})\,\mathrm{d}t' \\
+ \lim_{t\to1^-}\textstyle\int_0^t \big(2\beta_{1-t'}\big(\frac{\dot\alpha_{1-t'}}{\alpha_{1-t'}}\beta_{1-t'} - \dot\beta_{1-t'}\big) + \chi(1-t')\big)\exp\big(-\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\alpha_{1-t}}{\alpha_{1-t'}}\,\mathrm{d}B_{t'}.
\end{aligned} \tag{151}$$

The assumption on χ in (25) is equivalent, up to a rearrangement of the notation and a flip in the time variable, to the statement that for all $t' \in [0,1)$,

$$\lim_{t\to1^-}\exp\big(-\textstyle\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\alpha_{1-t} = 0. \tag{152}$$

Hence, under assumption (25), the factor accompanying $\vec X_0$ in equation (151) is zero. Moreover, this assumption also implies that

$$\begin{aligned}
&\lim_{t\to1^-}\alpha_{1-t}\textstyle\int_0^t \exp\big(-\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\chi(1-t')}{2\beta_{1-t'}^2}\xi_{1-t'}(\vec X_{t'})\,\mathrm{d}t' \\
&= \textstyle\int_0^1 \big(\lim_{t\to1^-}\exp\big(-\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\alpha_{1-t}\big)\frac{\chi(1-t')}{2\beta_{1-t'}^2}\xi_{1-t'}(\vec X_{t'})\,\mathrm{d}t' = 0.
\end{aligned} \tag{153}$$

If we plug (152) and (153) into (151), we obtain that

$$\vec X_1 = \lim_{t\to1^-}\textstyle\int_0^t \big(2\beta_{1-t'}\big(\frac{\dot\alpha_{1-t'}}{\alpha_{1-t'}}\beta_{1-t'} - \dot\beta_{1-t'}\big) + \chi(1-t')\big)\exp\big(-\int_{t'}^t \frac{\chi(1-s)}{2\beta_{1-s}^2}\,\mathrm{d}s\big)\frac{\alpha_{1-t}}{\alpha_{1-t'}}\,\mathrm{d}B_{t'}, \tag{154}$$

which shows that $\vec X_1$ is independent of $\vec X_0$. Next, we leverage that $\vec{\boldsymbol{X}}$ and $\boldsymbol{X}$ have equal distributions over trajectories (Proposition 4). In particular, the joint distribution of $(\vec X_0, \vec X_1)$ is equal to the joint distribution of (X_1, X_0). We conclude that X_1 and X_0 are independent, which is the definition of the memorylessness property. Hence, the assumption (25) is sufficient for memorylessness to hold.

It remains to prove that the assumption (25) is necessary. Looking at equation (150) we deduce that generally, for any $t \in [0,1)$, $\vec X_0$ and $\vec X_t$ are not independent, because the first two terms in (150) are different from zero. Thus, if there existed a $t' \in [0,1)$ such that the limit (152) is different from zero, then $\vec X_1$ would not be independent from $\vec X_{t'}$, which means that in general it would not be independent of $\vec X_0$ either.

D.2 Proof of Theorem 1: fine-tuning recipe for general noise schedules

The proof of this result relies heavily on the properties of the Hamilton-Jacobi-Bellman equation:

Theorem 3 (Hamilton-Jacobi-Bellman equation)**.** *If we define the infinitesimal generator*

$$\mathcal{L} := \tfrac{1}{2}\textstyle\sum_{i,j=1}^d (\sigma\sigma^\top)_{ij}(t)\partial_{x_i}\partial_{x_j} + \sum_{i=1}^d b_i(x,t)\partial_{x_i}, \tag{155}$$

the value function V for the SOC problem (12)-(13) solves the following Hamilton-Jacobi-Bellman (HJB) partial differential equation:

$$\begin{aligned}
&\partial_t V(x,t) = -\mathcal{L}V(x,t) + \tfrac{1}{2}\|(\sigma^\top\nabla V)(x,t)\|^2 - f(x,t), \\
&V(x,T) = g(x).
\end{aligned} \tag{156}$$

Consider forward SDEs like (63), starting from the distributions p^{base} and p^*, where $p^*(x) \propto p^{\text{base}}(x)\exp(r(x))$.

$$\mathrm{d}\vec{X}_t = \vec{b}(\vec{X}_t, t)\,\mathrm{d}t + \sigma(t)\,\mathrm{d}B_t, \qquad \vec{X}_0 \sim p^{\text{base}}, \tag{157}$$

$$\mathrm{d}\vec{X}^*_t = \vec{b}^*(\vec{X}^*_t, t)\,\mathrm{d}t + \sigma(t)\,\mathrm{d}B_t, \qquad \vec{X}_0 \sim p^*. \tag{158}$$

where the drifts are defined as

$$\begin{aligned}\vec{b}(x,t) &= -\kappa_{1-t}x + \big(\tfrac{\sigma(1-t)^2}{2} - \eta_{1-t}\big)\mathfrak{s}(x, 1-t) = -\kappa_{1-t}x + \big(\tfrac{\sigma(1-t)^2}{2} - \eta_{1-t}\big)\nabla \log \vec{p}_t(x),\\ \vec{b}^*(x,t) &= -\kappa_{1-t}x + \big(\tfrac{\sigma(1-t)^2}{2} - \eta_{1-t}\big)\mathfrak{s}^*(x, 1-t) = -\kappa_{1-t}x + \big(\tfrac{\sigma(1-t)^2}{2} - \eta_{1-t}\big)\nabla \log \vec{p}^*_t(x),\end{aligned} \tag{159}$$

and $\vec{p}_t$, $\vec{p}^*_t$ are the densities of X_t, $\vec{X}_t$, respectively. $\vec{p}_t$, $\vec{p}^*_t$ satisfy Fokker-Planck equations:

$$\begin{aligned}\partial_t \vec{p}_t &= \nabla\cdot(\vec{b}(x,t)\vec{p}_t) + \nabla\cdot(\tfrac{\sigma(1-t)^2}{2}\nabla\vec{p}_t), \qquad \vec{p}_0 = p^{\text{base}},\\ \partial_t \vec{p}^*_t &= \nabla\cdot(\vec{b}^*(x,t)\vec{p}^*_t) + \nabla\cdot(\tfrac{\sigma(1-t)^2}{2}\nabla\vec{p}^*_t), \qquad \vec{p}_0 = p^*.\end{aligned} \tag{160}$$

Plugging (159) into (160), we obtain

$$\begin{aligned}\partial_t \vec{p}_t &= \nabla\cdot(\kappa_{1-t}x\vec{p}_t) + \nabla\cdot\big(\eta_{1-t}\nabla\vec{p}_t\big), \qquad \vec{p}_0 = p^{\text{base}},\\ \partial_t \vec{p}^*_t &= \nabla\cdot(\kappa_{1-t}x\vec{p}^*_t) + \nabla\cdot\big(\eta_{1-t}\nabla\vec{p}^*_t\big), \qquad \vec{p}_0 = p^*.\end{aligned} \tag{161}$$

We apply the Hopf-Cole transformation to obtain PDEs for $-\log\vec{p}_t$ (and $-\log\vec{p}^*_t$ analogously):

$$\begin{aligned}-\partial_t(-\log\vec{p}_t) = \tfrac{\partial_t p_t}{p_t} &= \tfrac{\nabla\cdot(\kappa_{1-t}x\vec{p}_t) + \nabla\cdot\big(\eta_{1-t}\nabla\vec{p}_t\big)}{p_t}\\ &= \kappa_{1-t}\nabla\cdot x + \kappa_{1-t}\langle x, \nabla\log\vec{p}_t\rangle + \eta_{1-t}\tfrac{\nabla\cdot(\nabla\log\vec{p}_t\exp(\log p_t))}{p_t}\\ &= \kappa_{1-t}d + \kappa_{1-t}\langle x, \nabla\log\vec{p}_t\rangle + \eta_{1-t}\big(\Delta\log\vec{p}_t + \|\nabla\log\vec{p}_t\|^2\big).\end{aligned} \tag{162}$$

Hence, if we define $\mathscr{V}(x,t) = -\log\vec{p}_t(x)$, $\mathscr{V}^*(x,t) = -\log\vec{p}^*_t(x)$, then $\mathscr{V}$ and $\mathscr{V}^*$ satisfy the following Hamilton-Jacobi-Bellman equations:

$$-\partial_t\mathscr{V} = \kappa_{1-t}d - \kappa_{1-t}\langle x, \nabla\mathscr{V}\rangle + \eta_{1-t}\big(-\Delta\mathscr{V} + \|\nabla\mathscr{V}\|^2\big), \qquad \mathscr{V}(x,0) = -\log p^{\text{base}}(x), \tag{163}$$

$$-\partial_t\mathscr{V}^* = \kappa_{1-t}d - \kappa_{1-t}\langle x, \nabla\mathscr{V}^*\rangle + \eta_{1-t}\big(-\Delta\mathscr{V}^* + \|\nabla\mathscr{V}^*\|^2\big), \qquad \mathscr{V}^*(x,0) = -\log p^*(x). \tag{164}$$

Now, define $\hat{\mathscr{V}}(x,t) = \mathscr{V}^*(x,t) - \mathscr{V}(x,t)$. Subtracting (164) from (163), we obtain

$$\begin{aligned}-\partial_t\hat{\mathscr{V}} &= -\kappa_{1-t}\langle x, \nabla\hat{\mathscr{V}}\rangle + \eta_{1-t}\big(-\Delta\hat{\mathscr{V}} + \|\nabla\mathscr{V}^*\|^2 - \|\nabla\mathscr{V}\|^2\big)\\ &= -\kappa_{1-t}\langle x, \nabla\hat{\mathscr{V}}\rangle + \eta_{1-t}\big(-\Delta\hat{\mathscr{V}} + \|\nabla(\hat{\mathscr{V}} + \mathscr{V})\|^2 - \|\nabla\mathscr{V}\|^2\big)\\ &= -\kappa_{1-t}\langle x, \nabla\hat{\mathscr{V}}\rangle + \eta_{1-t}\big(-\Delta\hat{\mathscr{V}} + \|\nabla\hat{\mathscr{V}}\|^2 + 2\langle\nabla\mathscr{V}, \nabla\hat{\mathscr{V}}\rangle\big)\\ &= \langle -\kappa_{1-t}x + 2\eta_{1-t}\nabla\mathscr{V}, \nabla\hat{\mathscr{V}}\rangle + \eta_{1-t}\big(-\Delta\hat{\mathscr{V}} + \|\nabla\hat{\mathscr{V}}\|^2\big)\\ &= \langle -\kappa_{1-t}x - 2\eta_{1-t}\mathfrak{s}(x, 1-t), \nabla\hat{\mathscr{V}}\rangle + \eta_{1-t}\big(-\Delta\hat{\mathscr{V}} + \|\nabla\hat{\mathscr{V}}\|^2\big),\\ \hat{\mathscr{V}}(x,0) &= -\log p^*(x) + \log p^{\text{base}}(x) = -r(x) + \log\big(\textstyle\int p^{\text{base}}(y)\exp(r(y))\,\mathrm{d}y\big).\end{aligned} \tag{165}$$

Hence, $\hat{\mathscr{V}}$ also satisfies a Hamilton-Jacobi-Bellman equation. If we define V such that $\hat{\mathscr{V}}(x,t) = V(x, 1-t)$, we have that

$$\partial_t V = \langle -\kappa_t x - 2\eta_t\mathfrak{s}(x,t), \nabla V\rangle + \eta_t\big(-\Delta V + \|\nabla V\|^2\big), \qquad V(x,1) = r(x) - \log\big(\textstyle\int p^{\text{base}}(y)\exp(r(y))\,\mathrm{d}y\big). \tag{166}$$

Using Theorem 3, we can reverse-engineer V as the value function of the following SOC problem:

$$\min_{u\in\mathcal{U}} \mathbb{E}\big[\tfrac{1}{2}\textstyle\int_0^1 \|u(X^u_t, t)\|^2\,\mathrm{d}t - r(x) + \log\big(\int p^{\text{base}}(y)\exp(r(y))\,\mathrm{d}y\big)\big], \tag{167}$$

$$\text{s.t. } \mathrm{d}X^u_t = \big(\kappa_t x + 2\eta_t\mathfrak{s}(x,t) + \sqrt{2\eta_t}u(X^u_t, t)\big)\,\mathrm{d}t + \sqrt{2\eta_t}\mathrm{d}B_t, \qquad X^u_0 \sim p_0. \tag{168}$$